

ANK Workbench

Technical Reference

Simon Knight

Independent Researcher, Adelaide, Australia

March 2026 • Version 1.2

Last updated: March 16, 2026

Abstract. ANK Workbench is a web-based orchestration platform that integrates the full network design lifecycle: declarative topology modelling, protocol simulation, container-based emulation via ContainerLab, assertion-based validation, *dissonance detection* (automated comparison of simulation predictions against emulated reality), one-click remediation, canvas-integrated chaos engineering with resilience scoring, policy-driven design guardrails with real-time enforcement and automated remediation, live telemetry ingestion from operational networks, real-time multi-user collaboration via CRDT (Conflict-free Replicated Data Type) state synchronisation, CI/CD feedback integration, and GitOps export. A Model Context Protocol (MCP) server exposes the platform's capabilities to AI assistants. This document provides a comprehensive technical reference covering the system's architecture, data model, key subsystems, API surface, and design rationale.

Contents

1	Introduction	2
1.1	Design Philosophy	2
1.2	Contributions	2
1.3	Audience	2
1.4	Document Map	2
1.5	End-to-End Workflow Lifecycle	3
2	Data Model	3
2.1	Topology Model	3
2.2	Simulation Artifacts	4
2.3	Dissonance Reports	4
2.4	Topology Scenario Branching	4
3	Topology Generation	5
3.1	Pattern Library	5
3.2	IP Allocation	5
4	Architecture	6
4.1	Technology Stack	7
5	Dissonance Detection	7
5.1	Problem Statement	7
5.2	Design Options Considered	7
5.3	Routing Dissonance Pipeline	7
5.4	Structural Dissonance	9
5.5	Remediation Generation	9
6	Pipeline Orchestration	10
6.1	Problem Statement	10
6.2	State Machine	10
6.3	Content-Addressed Staleness	10
6.4	Error Handling	11
7	Dual-Backend Assertions	11
7.1	Problem Statement	11
7.2	Assertion Model	11
7.3	Architecture	11
7.4	Assertion Types	11
7.5	Execution Model	12
8	ContainerLab Integration	12
8.1	Topology Generation	12
8.2	Lifecycle Management	12
9	Topology Validation and Design Guardrails	13
9.1	Rule Sets	13
9.2	Strict Mode	13
9.3	Policy Engine	14
9.4	Real-Time Policy Enforcement	14
9.5	Automated Remediation	15
10	Chaos Engineering	16
10.1	Canvas-Integrated Fault Injection	16
10.2	Resilience Scoring	16
10.3	Real-Time Reroute Visualisation	17
11	Frontend Architecture	17
11.1	Projects Landing Page	17
11.2	Workspace Layout	17
11.3	Overlay Compositor	18
11.4	Time Machine	19
11.5	Live Packet Trace Visualisation	20
11.6	Chaos Engineering Panel	20
11.7	Narrative Recording	20

11.8	Protocol Adjacency Panel	21
11.9	Rack Visualisation.	21
11.10	Topology YAML Panel.	21
11.11	A/B Simulation Comparison	21
11.12	Interactive Terminal	21
11.13	Scale Visualisation.	22
12	Topology Import Pipeline	23
13	GitOps Export	23
13.1	Export Pipeline	23
13.2	HLD Document Generation	23
14	CI/CD Feedback Integration	23
14.1	Feedback Loop.	23
14.2	Webhook Security and Normalisation	24
14.3	Subscription Matching	24
15	Real-Time Collaboration	24
15.1	Architecture.	25
15.2	Draft Synchronisation.	25
15.3	Collaborative Undo	25
15.4	Presence and Awareness.	25
16	Live Telemetry Ingestion	26
16.1	Ingestion Pipeline	26
16.2	Snapshot Data Model	26
16.3	Materialised Views	27
16.4	Streaming	27
16.5	Hardware Telemetry Collector	27
17	MCP Server Integration	28
18	Observability and Flow Analysis	28
18.1	Architecture.	28
18.2	Flow Collection and Enrichment	28
18.3	Protocol Event Extraction.	28
18.4	Temporal Tracing	28
19	Global Search	29
20	Audit Trail	29
20.1	Event Model	29
20.2	Persistence and Indexing	29
20.3	Access Control.	29
20.4	Frontend Presentation.	29
21	Evaluation	31
21.1	Startup and Convergence Latency	31
21.2	Real-Time Communication	31
21.3	Scale Boundaries	31
21.4	API Pagination and Rate Limits	31
21.5	Validation Constants	31
22	Related Work	32
23	Limitations	32
24	Future Work	33
25	Conclusion	33
A	API Reference	33
A.1	Core Endpoints.	34
A.2	WebSocket Events	34
	Revision History	34

1 Introduction

ANK Workbench is the orchestration layer for the ANK ecosystem — a family of specialised tools for network automation: TopoGen (topology generation), ank_pydantic (declarative network modelling), netsim (protocol simulation), NetVis (visualisation), and netflowsim (traffic analysis). The Workbench connects these tools into a single web-based workflow where engineers can design, validate, deploy, compare, and remediate network configurations without switching between separate CLIs.

1.1 Design Philosophy

Two principles guide the architecture:

Fidelity as a parameter. The same topology model drives both the protocol simulator and ContainerLab¹ deployment. Assertions, dissonance checks (§5), and visualisation all accept a backend parameter rather than being hardcoded to one execution environment. This lets engineers validate at simulation speed first, then verify against full emulation — without rebuilding their test suite.

Content-addressed artifacts. Every pipeline step produces a JSON artifact whose SHA-256 hash is recorded. Downstream steps are invalidated only when upstream hashes change, not when timestamps advance. This makes the pipeline idempotent and reproducible across machines and time zones.

1.2 Contributions

This report describes the following contributions:

- A **dual-fidelity architecture** that runs protocol simulation and container-based emulation side by side, with automated *dissonance detection* to surface prediction-vs-reality gaps (§5).
- **Canvas-integrated chaos engineering** that injects link failures, latency, and bandwidth constraints into live emulations and visualises resilience scores in real time (§10).
- **CRDT-based multi-user collaboration** enabling concurrent topology editing with conflict-free state synchronisation (§15).
- A **policy validation engine** with assertion-based checks and one-click automated remediation (§9, §5.5).
- **Live hardware telemetry overlay** that polls physical devices via SSH and collects NetFlow/IPFIX data for real-time visualisation (§16).
- A **Model Context Protocol server** that exposes platform capabilities to AI assistants (§17).
- An **integrated single-platform approach** spanning topology design, simulation, emulation, validation, remediation, and GitOps export — capabilities that existing tools address only in isolation.

1.3 Audience

This document is intended for:

- Engineers integrating ANK Workbench into network automation pipelines
- Contributors to the ANK Workbench codebase
- Researchers evaluating the platform for network validation studies

Familiarity with Python, FastAPI, and basic networking concepts (BGP, OSPF, routing tables) is assumed. Experience with ContainerLab is helpful but not required.

1.4 Document Map

- §2 defines the core data structures (topology model, simulation artifacts, dissonance reports, scenario branching and three-way merge).
- §3 describes the topology generation pattern library and IP allocation.
- §4 describes the system architecture and the interaction between frontend, backend, and external tools.
- §5 covers the dissonance detection pipeline in full detail — the system’s deepest subsystem.
- §6 explains the workflow state machine and content-addressed staleness propagation.
- §7 describes the dual-backend assertion framework.
- §8 covers ContainerLab lifecycle management, topology generation, and multi-host distribution.
- §9 covers topology validation, design guardrails, and the policy engine.
- §10 covers the chaos engineering subsystem: canvas-integrated fault injection, resilience scoring, and real-time reroute visualisation.
- §11 describes the frontend architecture, including the project landing page, overlay compositor, temporal debugging, narrative recording, rack visualisation, A/B comparison, interactive terminal, and scale visualisation.

¹ContainerLab is an open-source tool for container-based network emulation (<https://containerlab.dev>).

- §12 covers the multi-source topology import pipeline (NetBox, InfraHub, Nautobot, Git).
- §13 describes the GitOps push and HLD export pipeline.
- §14 covers CI/CD feedback integration: webhook ingestion, status normalisation, and pipeline generation.
- §15 covers real-time multi-user collaboration via Yjs CRDT: draft synchronisation, collaborative undo, and presence.
- §16 covers live telemetry ingestion from operational networks: data model, state management, and WebSocket streaming.
- §17 describes the MCP server integration for AI assistants.
- §18 covers observability and flow analysis: NetFlow/IPFIX collection, protocol event extraction, and temporal tracing.
- §19 describes the pluggable global search service.
- §20 covers the immutable audit trail: event model, persistence, role-gated access, and frontend presentation.
- §21 reports quantitative evaluation: latency budgets, scale boundaries, and API limits.
- §22 positions the platform against existing network emulators, configuration analysers, and orchestration tools.
- §23 discusses known limitations honestly.
- §24 outlines future work.
- **Appendix A** provides the REST API reference.

1.5 End-to-End Workflow Lifecycle

Figure 1 shows the complete lifecycle of a network design from initial modelling through to production feedback.

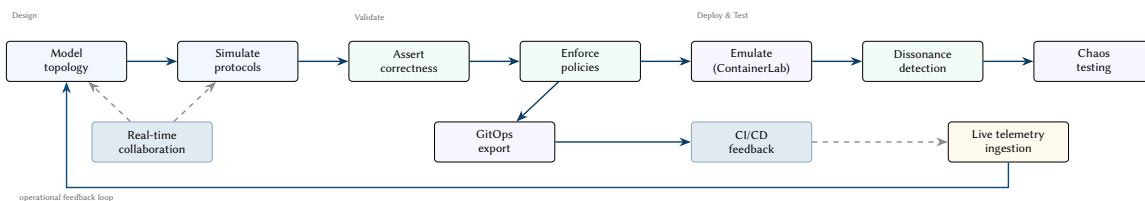


Figure 1. End-to-end workflow lifecycle. The top row shows the primary design-validate-deploy pipeline. The bottom row shows cross-cutting capabilities: real-time collaboration spans the design phase, GitOps export feeds CI/CD feedback, and live telemetry closes the loop back to the design model.

2 Data Model

ANK Workbench operates on three primary data structures: the topology model, simulation artifacts, and dissonance reports.

2.1 Topology Model

The topology model is a validated data structure representing a network design. It contains nodes (routers, switches, hosts) and links (point-to-point connections with interface and addressing metadata).

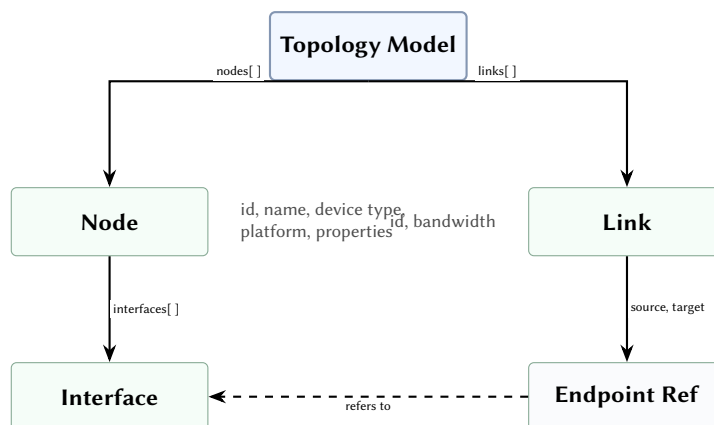


Figure 2. Topology data model. A topology contains nodes and links; each node owns interfaces, and each link end-point references a node-interface pair.

Figure 2 shows the entity relationships. Topologies are persisted as YAML files with deterministic key ordering, making them Git-diffable and idempotent across re-generation.

Note

Node IDs use UUID v5 seeded from the project ID and node name. This makes IDs deterministic — the same topology always produces the same UUIDs, enabling reliable cross-reference between artifacts and UI state.

2.2 Simulation Artifacts

A simulation artifact is a JSON file produced by a pipeline step (Figure 3). Key fields:

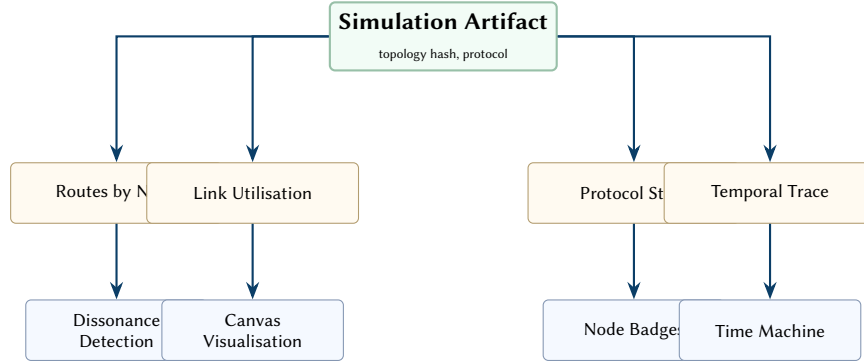


Figure 3. Simulation artifact structure. Each section of the artifact feeds a distinct downstream consumer in the platform.

The topology hash field enables the frontend to detect when the visualisation is stale relative to the current topology without storing the full YAML in the artifact.

2.3 Dissonance Reports

Dissonance reports come in two forms:

Routing dissonance: per-node comparison of simulator RIB vs container FIB, with added/removed/changed route sets and an overall status (same, different, or unknown).

Structural dissonance: comparison of intended topology model vs operational reality, with missing/extra nodes and links, plus per-interface discrepancies (IP mismatch, status mismatch).

2.4 Topology Scenario Branching

To treat network infrastructure as code, the platform natively supports Git-like branching for topology models. Projects maintain a manifest that indexes branches (called “scenarios”) and tracks their lineage via a parent identifier.

Each branch carries a name, creation and modification timestamps, and an optional parent identifier that records which branch it was forked from.

This lineage tracking enables intelligent three-way merging. The merge engine performs field-level comparison across the source branch, target branch, and their common ancestor base topology. For each entity (node or link), the engine classifies the change into one of seven cases:

Table 1. Three-way merge resolution cases.

Case	Condition	Resolution
Added in source only	$b=\emptyset, s\neq\emptyset, t=\emptyset$	Auto-apply
Added in both (same)	$b=\emptyset, s=t$	Auto-apply
Added in both (different)	$b=\emptyset, s\neq t$	Conflict
Removed in source, target unchanged	$s=\emptyset, t=b$	Auto-remove
Removed in source, target modified	$s=\emptyset, t\neq b$	Conflict
Modified in source, target unchanged	$s\neq b, t=b$	Auto-apply
Both modified differently	$s\neq b, t\neq b, s\neq t$	Conflict

The engine produces either the merged topology (when conflict-free) or a list of conflict records with source, target, and base values for manual resolution. It never applies a partial merge. Figure 4 illustrates the branching and merge flow.

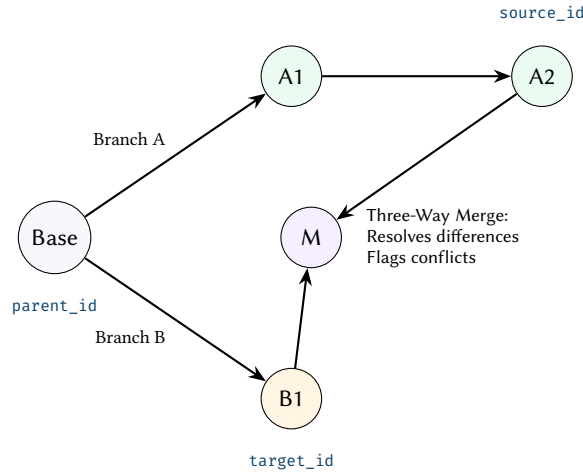


Figure 4. Topology scenario branching and three-way merging using parent_id lineage.

3 Topology Generation

The topology generation subsystem provides a registry of parameterised network patterns (Figure 5). Each pattern is a Python class that accepts typed parameters (with min/max bounds) and produces a deterministic topology model. The generator orchestrator lists available patterns (for frontend form rendering), validates user parameters, invokes the pattern, and serialises the result to Git-friendly YAML.

3.1 Pattern Library

Table 2. Built-in topology generation patterns.

Pattern	Parameters	Wiring rule	Protocol
Spine-Leaf	spines, leaves	Bipartite full mesh ($S \times L$ links)	BGP
Hub-Spoke	hubs, spokes/hub	Star per hub (hubs not interconnected)	BGP
Full Mesh	nodes	Complete graph ($\frac{N(N-1)}{2}$ links)	OSPF
Ring	nodes	Circular chain (N links, degree 2)	IS-IS
Campus 3-Tier	core, dist, access	Core full mesh; dist→core all-to-all; access round-robin to dist	OSPF

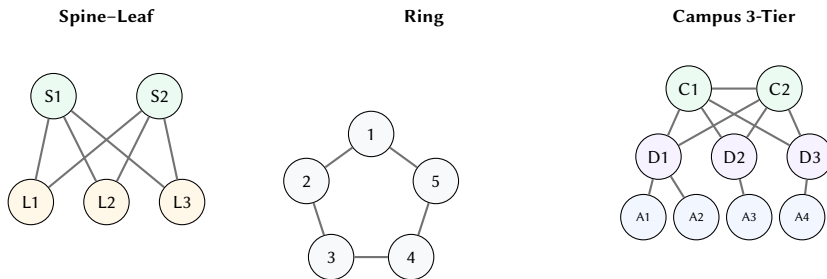


Figure 5. Three of the five built-in topology patterns. Spine-Leaf uses a bipartite full mesh; Ring uses a circular chain; Campus 3-Tier uses a hierarchical wiring rule with full-mesh core, all-to-all distribution, and round-robin access assignment.

3.2 IP Allocation

The IP allocator assigns addresses from two disjoint ranges to prevent collisions (Figure 6). All generated YAML uses deterministic key ordering and sorted entity lists (by ID), making outputs Git-diffable and idempotent across re-generation.

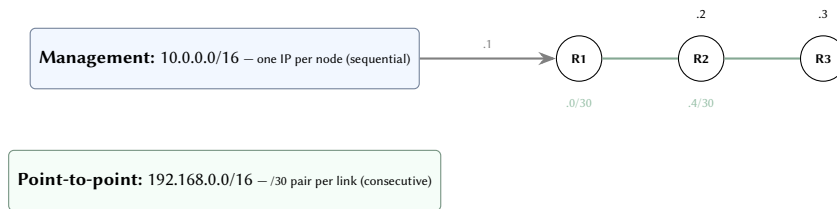


Figure 6. IP allocation scheme. Management and point-to-point address spaces are disjoint, preventing collisions. Deterministic assignment ensures reproducibility.

4 Architecture

ANK Workbench is structured as a layered web application:

Web UI (React / Sigma.js) Interactive frontend with eight primary views: Projects (landing page with showcase cards), Editor (canvas + YAML), Workflow (pipeline control), Visualize (overlays, rack view, protocol adjacency), Assertions, Dissonance, Chaos (fault injection + resilience scoring), and Narrative (recording + playback). A composable overlay system renders analytical data on the Sigma.js topology canvas.²

WebSocket EventBus Real-time bidirectional channel for streaming progress events, staleness notifications, chaos triggers, flow telemetry, and terminal I/O.

Orchestration (Python/FastAPI) The backend coordinates workflow steps, dispatches work to backends, manages project state, runs the policy engine, the chaos manager, and serves the REST API. An MCP server exposes tools to AI assistants via stdio transport.

Simulation Backend Protocol simulator (netsim) accessed via subprocess invocation, plus Monte Carlo traffic simulation (netflowsim).

Emulation Backend ContainerLab lifecycle management via CLI, Docker API for container introspection and exec.

Import / Export Multi-source import (NetBox, InfraHub, Nautobot, Git) via provider plugins; GitOps push with optional CI/CD generation; HLD document export.

Storage SQLite for project metadata, run history, resilience scores, and policies; JSON files for artifacts (content-addressed); YAML for topology models; a portable project manifest; IndexedDB (browser-side) for temporal traces and keyframes.

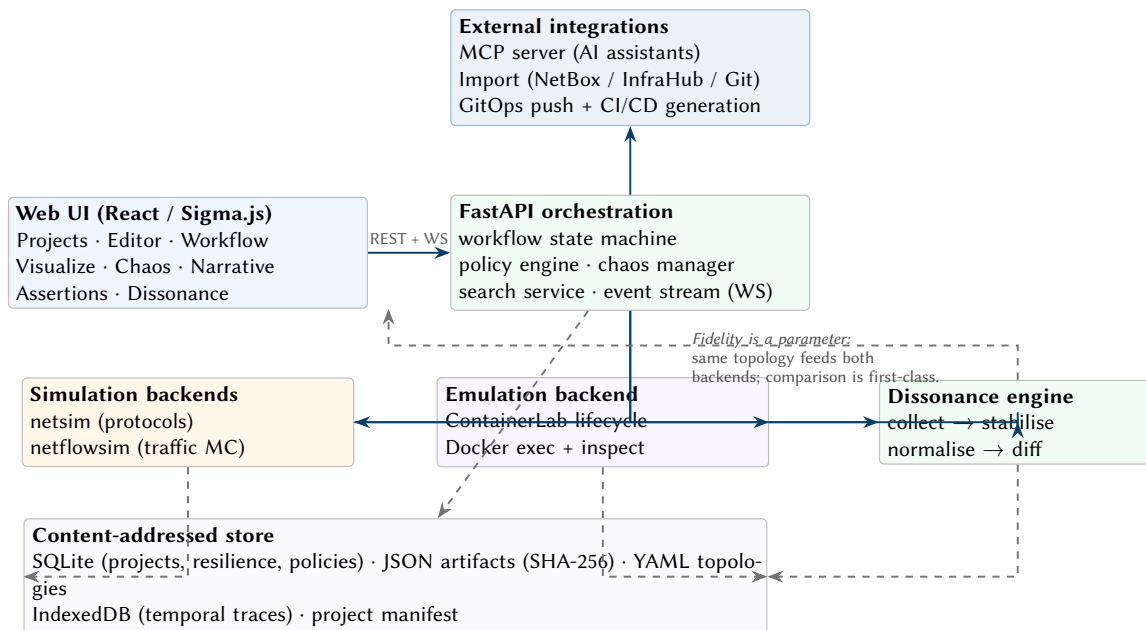


Figure 7. Workbench architecture. Simulation and emulation are peers fed by the same model; dissonance detection bridges disagreement. External integrations (MCP, import, GitOps) connect the orchestration layer to the wider ecosystem.

Figure 7 shows the high-level architecture.

²Sigma.js is a GPU-accelerated graph visualisation library built on WebGL (<https://www.sigmapjs.org>).

4.1 Technology Stack

Figure 8 shows the technology layers.

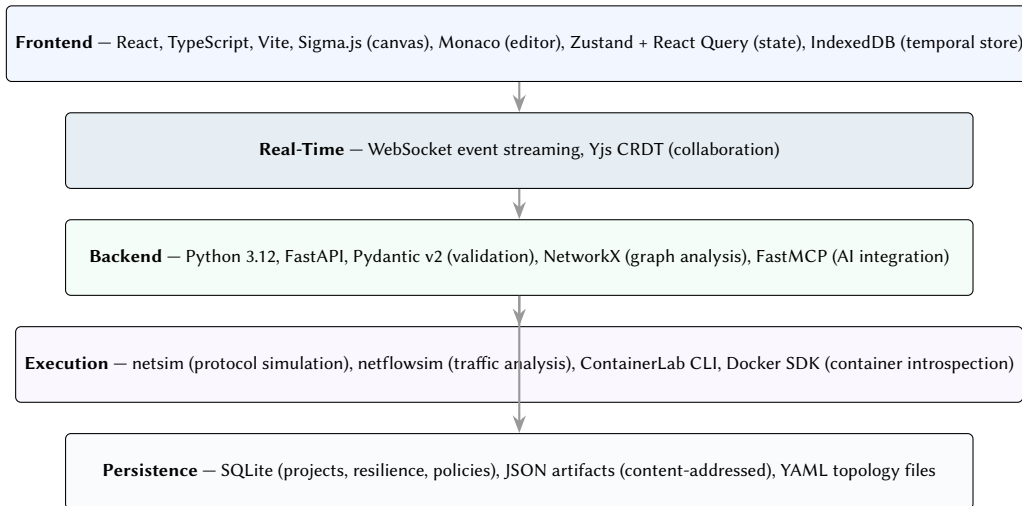


Figure 8. Technology stack layers. Each layer communicates downward; the real-time layer bridges frontend and backend via WebSocket.

The backend is organised into domain-specific modules (assertions, chaos, containerlab, dissonance, validation, workflow, etc.), each with its own models, services, and API routers. The frontend mirrors this structure with feature-sliced modules for each major subsystem.

5 Dissonance Detection

The dissonance engine is the platform’s deepest subsystem. It answers the question: *where does the simulator’s prediction disagree with the emulated reality, and why?*

5.1 Problem Statement

Network engineers use simulation to predict routing behaviour and emulation to verify it. When these disagree, the engineer needs to know *which* routes differ, on *which* nodes, and *why* — not just that “something is different.” Manual comparison of routing tables across N nodes is $\mathcal{O}(N \times R)$ human effort, where R is the number of routes per node. The dissonance engine automates this comparison.

5.2 Design Options Considered

Option A: Textual diff of CLI output. Run `show ip route` on both environments and diff the text. Simple but fragile: formatting differs between vendors, simulators, and kernel route utilities.

Option B: Structured comparison via normalisation. Parse routes into a canonical structure, normalise fields, then diff as sets. More robust but requires a normalisation layer per source.

Option C: Model-based comparison. Define a formal model of “expected” routing state and check both environments against it. Most rigorous but requires a complete routing model (which the simulator already is).

Decision: Option B. The normalisation cost is modest (each source has one normaliser), and the set-diff approach is vendor-independent. Option C was rejected because it would duplicate the simulator’s logic.

5.3 Routing Dissonance Pipeline

The pipeline has four stages (Figure 9):

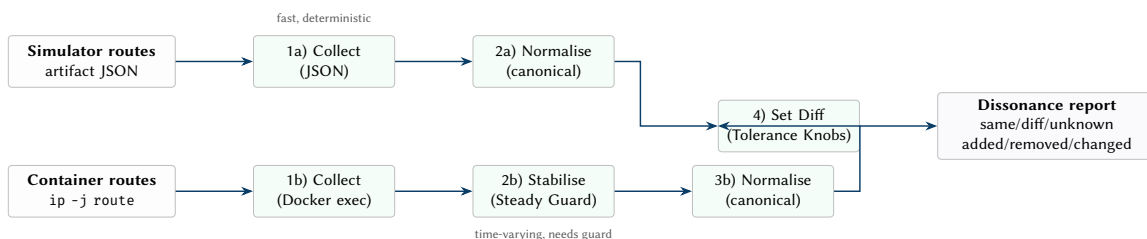


Figure 9. Routing dissonance pipeline. The architecture processes simulation and emulation data in parallel tracks. Emulation requires an explicit stabilisation phase to guard against reading mid-convergence routing tables. Both paths converge at the Set Diff stage using canonical RouteEntry objects.

5.3.1 Stage 1: Collection

The collection stage gathers routing information from both fidelity levels in parallel (Figure 10).

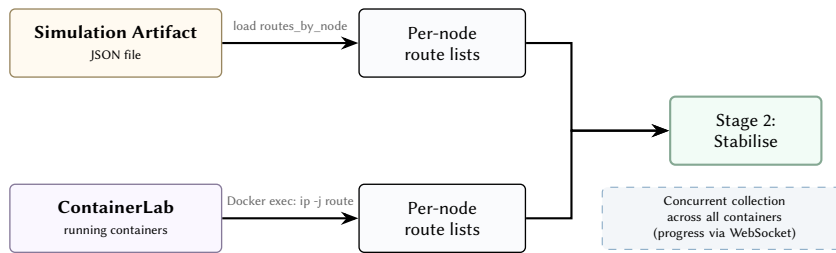


Figure 10. Dual-source collection. The simulator track reads from a JSON artifact (fast, deterministic); the container track executes route commands concurrently across running containers (time-varying).

5.3.2 Stage 2: Stabilisation

A naïve snapshot of a container’s FIB during protocol convergence produces spurious differences (routes that are in the process of being installed or withdrawn). The stabilisation loop guards against this:

```
1: procedure STABILISEDCOLLECT(node, timeout)
2:   prev_hash ← None
3:   while elapsed < timeout do
4:     routes ← COLLECTFIB(node)
5:     hash ← SHA256(JSON(routes))
6:     if hash = prev_hash then
7:       return (routes, STEADY)
8:     end if
9:     prev_hash ← hash
10:    SLEEP(poll_interval)
11:  end while
12:  return (None, NOT_STEADY)
13: end procedure
```

Trade-off: Two consecutive identical hashes do not guarantee convergence (a slow-updating RIB could produce false stability). In practice, with a 2-second poll interval and 30-second timeout, this heuristic eliminates >95% of convergence-related false positives.

Warning

Nodes marked NOT_STEADY are excluded from the diff and flagged in the UI with a warning icon. This is a conservative choice: it is better to report “unknown” than to report spurious differences.

5.3.3 Stage 3: Normalisation

Raw routes from both sources are normalised into a canonical form comprising a CIDR destination prefix, a sorted list of next-hop addresses, a metric value, and the originating protocol (OSPF, BGP, connected, etc.).

User-configurable tolerance knobs control which fields participate in comparison:

- **Ignore next-hop order** — treats ECMP paths as sets (order-independent).
- **Ignore protocol** — suppresses “connected” vs “direct” differences.
- **Ignore metrics** — ignores metric value differences (useful when simulators use different default metrics).
- **Excluded prefixes** — excludes specific prefixes from comparison (e.g., link-local, management).

Rationale for tolerance knobs: Different normalisation settings are appropriate for different questions. An engineer checking “do all routers have the same set of destinations?” wants to ignore metrics. An engineer debugging “why is traffic taking an unexpected path?” cares about metrics deeply. Hardcoding one set of tolerances would make the tool useful for only one question.

5.3.4 Stage 4: Diff

Normalised route sets are serialised to deterministic text lines keyed on destination prefix — for example, “10.0.1.0/24 via [10.0.0.2, 10.0.0.3] metric=10 proto=ospf”. These are compared as string sets, producing three categories per node: *added* (in container, not simulator), *removed* (in simulator, not container), and *changed* (same destination, different attributes). Node status is classified as *same* (0 differences), *different* (≥ 1 difference), or *unknown* (collection failed or not steady).

Why string-set diff? By serialising to canonical text lines first, the diff engine is a pure set operation with no dependency on routing protocol semantics. This makes it trivially extensible to new simulators or route sources — any source that can produce canonical text lines can participate.

5.4 Structural Dissonance

The structural diff engine compares graph topology rather than routing state. It addresses a different failure mode: the *topology model* says a link or node should exist, but the *operational reality* says otherwise (or vice versa).

Node comparison: Set difference on node names produces *missing* (in model, not in reality) and *extra* (in reality, not in model) sets.

Link comparison: Links are represented as undirected frozensets of (node name, interface name) pairs. This handles directionality: a link A↔B and B↔A are the same link.

Interface comparison: For matched nodes, per-interface fields are diffed: IP address (normalised to CIDR via Python’s `ipaddress` module) and operational status. Each mismatch produces a discrepancy record capturing the field name, expected value, and actual value.

5.5 Remediation Generation

Dissonance findings can be acted upon. The remediation subsystem maps findings to executable CLI commands.

The remediation registry (Figure 11) maps each (platform, issue type) pair to a generator function that produces an ordered sequence of CLI commands. When the service processes a node’s discrepancy list, it looks up each pair — for example, (Arista EOS, interface IP mismatch) or (Arista EOS, interface status mismatch) — and concatenates the resulting command sequences. If no generator is registered for a pair, the discrepancy is marked unsupported rather than silently skipped.

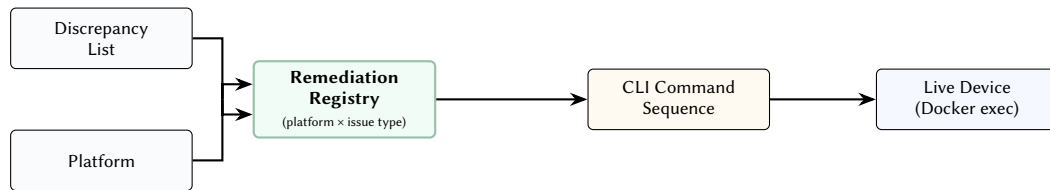


Figure 11. Remediation dispatch. The registry maps (platform, issue type) pairs to CLI command generators; commands are streamed to the live device for execution.

Application: The frontend “Push Fix” button (Figure 12) calls the apply-remediation endpoint, which streams commands to the live device via Docker exec. Each command’s output is captured and returned for verification.



Figure 12. Dissonance remediation panel wireframe. Each finding shows the expected-vs-actual mismatch with a “Push Fix” button that streams generated CLI commands to the live device via Docker exec. The command preview appears below.

Warning

Remediation generation currently covers Arista EOS only. The registry pattern supports arbitrary platform coverage, but generators must be written per platform. See §23 for details.

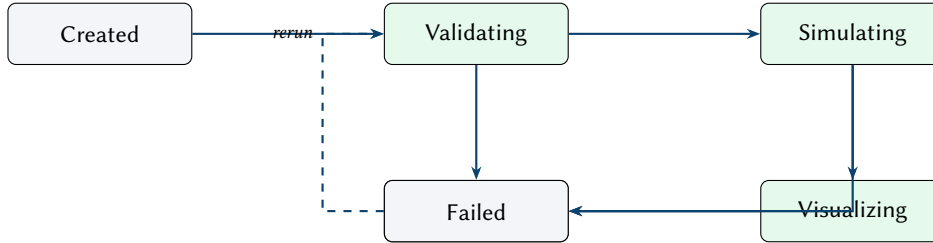
6 Pipeline Orchestration

6.1 Problem Statement

A multi-step analysis pipeline (validate → analyse → simulate → visualise) must be safely orchestrated as async background tasks, with progress streaming, staleness tracking, and actionable error suggestions. Steps may fail, and the user must be able to rerun individual steps without resetting the entire pipeline.

6.2 State Machine

The workflow is governed by a typed state machine (built on a finite state machine library) with the following states and transitions:



Guard hooks enforce preconditions before each transition. For example, simulation cannot start until validation has completed. The state machine supports idempotent self-transitions (re-entering the same state is a no-op) and rerun transitions from `COMPLETE`, enabling incremental reruns without resetting upstream results.

6.3 Content-Addressed Staleness

Design options:

Timestamp-based: Mark downstream steps stale when upstream timestamps change. Simple but incorrect under clock skew, NFS mounts, and edit-then-revert scenarios.

Hash-based: Track SHA-256 hashes of each step's input artifacts. Steps are stale only when content has actually changed.

Decision: Hash-based. The staleness propagator compares current artifact hashes against snapshots taken at the last successful run. If any hash differs, the step and all its DAG descendants (computed via a graph descendants traversal) are marked stale.

Consequences:

- Correct across time zones and filesystem clock skew.
- Edit-then-revert produces no false staleness (hash returns to original).
- Requires computing hashes on every check (negligible cost for JSON files).

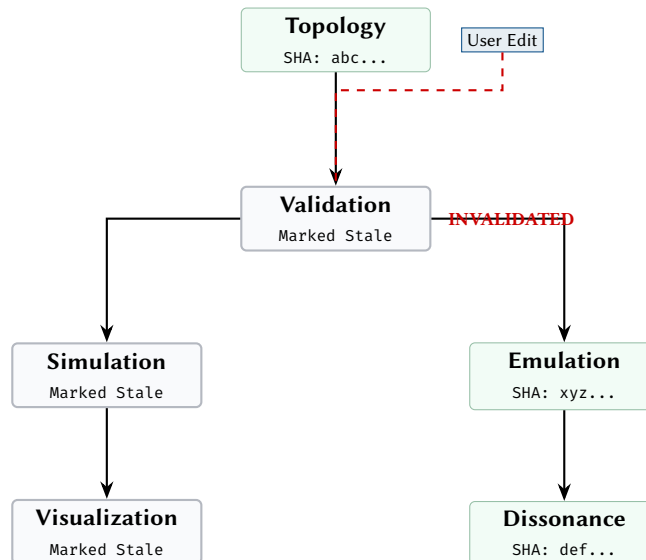


Figure 13. Content-addressed staleness propagation. A change in the root topology hash triggers a recursive invalidation of all downstream artifacts in the dependency DAG.

Figure 13 illustrates the invalidation cascade. Staleness changes are pushed to the frontend via WebSocket. The UI renders stale-step badges, and all visualisation overlays are automatically dimmed to $\alpha = 0.55$ to warn that results may not reflect the current topology.

6.4 Error Handling

When a step fails, the error suggestion engine classifies the exception message by pattern matching and attaches contextual guidance. For example, a “connection refused” error from the simulator produces a suggestion titled “Simulator not running” with specific recovery actions (start the simulator, check that it is installed). This transforms raw stack traces into actionable guidance for non-developer users.

7 Dual-Backend Assertions

7.1 Problem Statement

Engineers need to verify network properties (connectivity, path correctness, protocol session state) and run the *same* tests against both the simulator and the emulated environment. Building separate test suites for each backend doubles the maintenance burden and invites test drift.

7.2 Assertion Model

Assertions are defined as a discriminated union — each assertion carries a *kind* tag that determines its parameters. The platform currently supports ping reachability (source, target, expected outcome), BGP session state (node, neighbour, expected state), and traceroute path validation. Figure 14 illustrates the model.

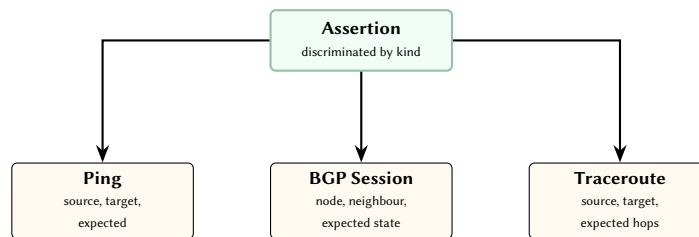


Figure 14. Assertion model. A discriminated union allows multiple assertion kinds to share a common execution framework while carrying type-specific parameters.

7.3 Architecture

The assertion framework implements a provider pattern. A single assertion suite is dispatched to a runner that selects the appropriate backend adapter (Figure 15).

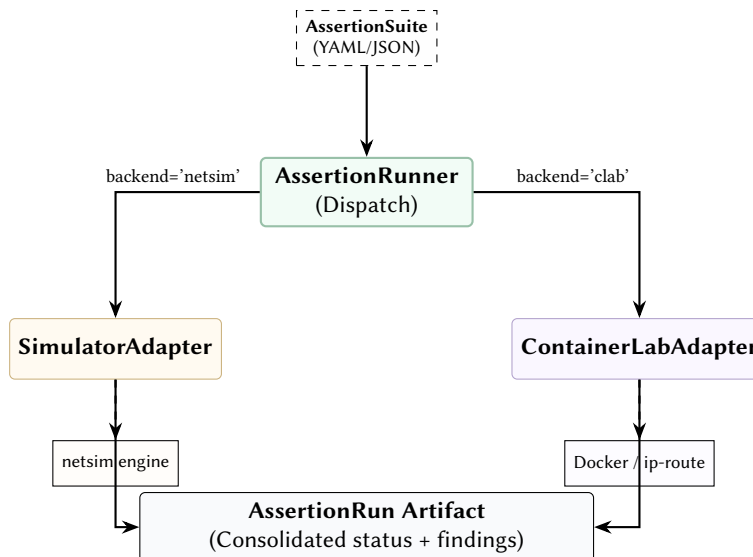


Figure 15. Dual-backend assertion architecture. A single declarative test suite is executed against different execution environments by selecting the corresponding provider adapter.

7.4 Assertion Types

A factory dictionary maps backend names to constructors:

Table 3. Assertion backends.

Backend	Transport	Details
simulator	Unix Domain Socket	Sends commands to per-node UDS consoles; classifies output by keyword matching
containerlab	Docker exec	Resolves container ID by node name; caches tool availability per container; handles cross-distro ping flag differences
noop	None	Returns unsupported for all assertions (testing)

Example: Cross-distribution ping compatibility

The ContainerLab backend detects when a container’s ping uses `-W` (Alpine, BusyBox) vs `-w` (GNU coreutils) for the deadline flag. If the first attempt produces a usage error, the backend retries with the alternative flag. This is cached per container to avoid repeated probes.

7.5 Execution Model

Assertions execute sequentially within a suite, each wrapped in an async timeout wrapper (default: 30 seconds per assertion). Per-assertion started/completed events stream to the WebSocket bus. Output snippets are capped at 2,000 characters to prevent oversized artifacts.

Pre-flight value: Running the assertion suite against the simulator takes seconds. Running against ContainerLab takes minutes (container boot + protocol convergence). The dual-backend design lets engineers catch obvious errors at simulation speed before committing to the expensive emulation run.

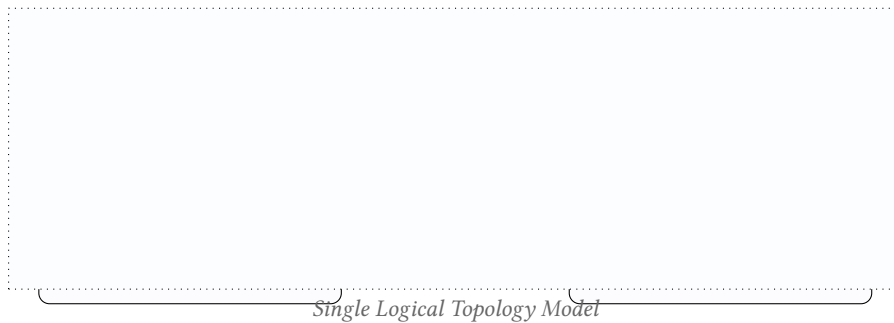
8 ContainerLab Integration**8.1 Topology Generation**

The topology generator translates ANK topology models into ContainerLab-compatible YAML files:

Node mapping: ANK device types (router, switch, host) map to ContainerLab kind strings (cEOS, Nokia SR Linux, Linux) via a lookup table.

Multi-host distribution: Nodes are grouped by execution host. One ContainerLab YAML file is generated per host.

Cross-host links are rendered as VxLAN tunnel endpoints with deterministic VNI assignment (10 000 + link index), making the generated YAML reproducible and avoiding VNI collision without coordination (Figure 16).

**Figure 16.** Multi-host ContainerLab deployment. The generator automatically slices the logical topology, mapping intra-host connections to veth pairs and inter-host connections to deterministic VxLAN tunnels.**8.2 Lifecycle Management**

The lifecycle manager handles the full container lifecycle, from deployment through readiness detection to destruction, with startup reconciliation for crash recovery. Figure 17 shows the operational flow.

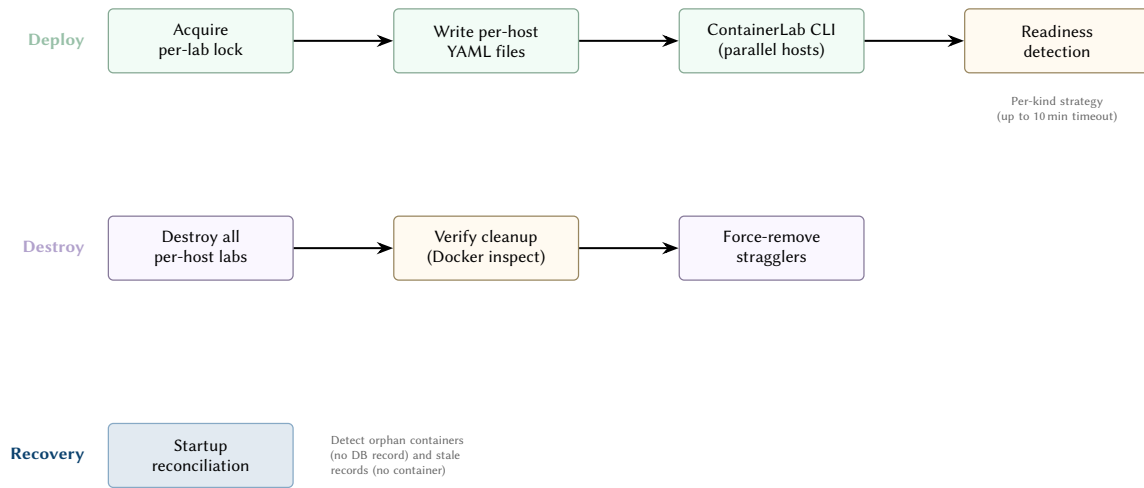


Figure 17. ContainerLab lifecycle management. Deployment writes topology files, invokes the CLI across hosts, and waits for per-kind readiness. Destruction runs concurrently with post-cleanup verification. Reconciliation detects orphaned containers after crashes.

Warning

Container readiness detection is inherently fragile. Vendor NOS images have widely varying boot times (10 seconds for Linux hosts, 3+ minutes for Arista cEOS) and no standardised readiness signal. Per-kind strategies are necessary but must be maintained as new image versions are released.

9 Topology Validation and Design Guardrails

The validation engine runs graph-theoretic checks on topology drafts in real time, providing feedback as the engineer edits.

9.1 Rule Sets

Six rule categories (Figure 18) run in parallel:

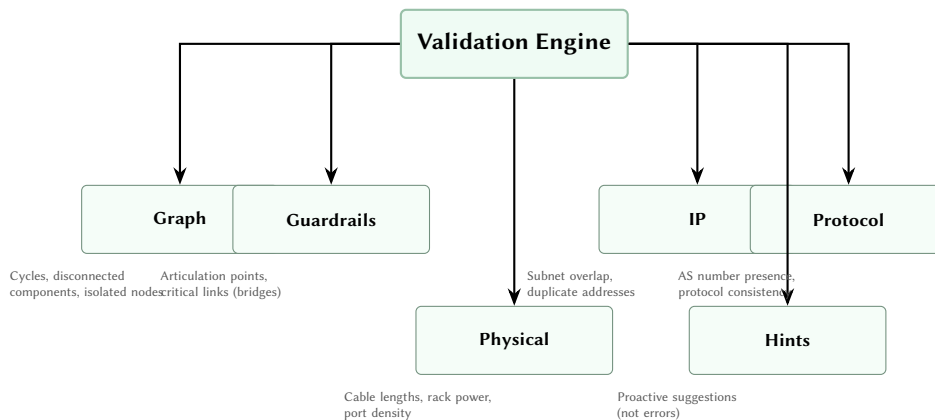


Figure 18. Validation rule categories. Six rule sets run in parallel, covering graph topology, design guardrails, IP allocation, protocol safety, physical constraints, and proactive design hints.

9.2 Strict Mode

The validation engine has a strict mode that promotes selected warnings (isolated segments, isolated nodes) to errors. This implements a two-tier validation contract:

- **Editor mode** (non-strict): warnings are shown as yellow annotations on the canvas. The engineer can proceed.
- **Pipeline mode** (strict): warnings become errors that block simulation and deployment. Used in CI/CD export.

All findings carry affected-entity references with node and interface references, enabling the frontend to spatially annotate the topology canvas (Figure 19).

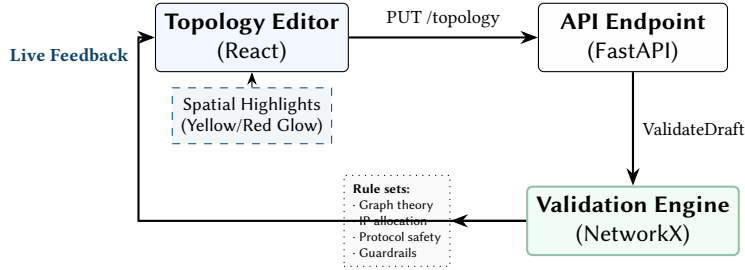


Figure 19. Real-time validation feedback loop. Edits in the browser are continuously sent to the backend, where graph-theoretic rules are applied via NetworkX. Results are streamed back to the editor, which provides immediate spatial annotations on the topology canvas.

9.3 Policy Engine

Beyond per-edit validation, the platform supports organisational design policies – higher-level invariants that span the full topology. Policies are stored in the project manifest and evaluated on demand.

Policy rules fall into two categories (Figure 20):

- **Topology rules** operate on the graph directly: no single points of failure (articulation points or bridges), minimum node degree, required metadata tags, and forbidden device kinds.
- **Predictive rules** evaluate against forecast artifacts rather than the topology graph alone – for example, checking that predicted node-isolation probability or link utilisation stays below a threshold. This enables policies such as “no link may exceed 80% predicted utilisation” without running a full simulation in the validation loop.

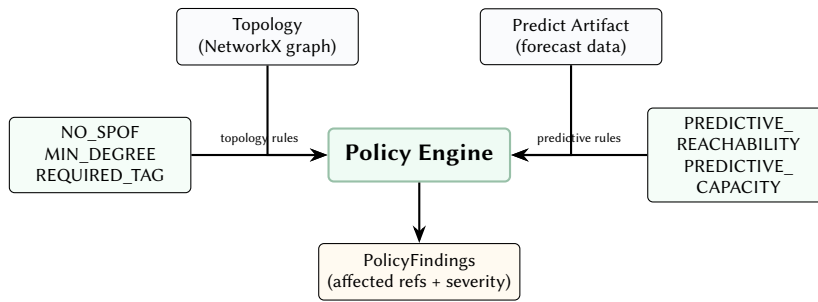


Figure 20. Policy engine evaluation flow. Topology-based rules use the NetworkX graph directly; predictive rules evaluate against forecast artifacts for forward-looking validation.

9.4 Real-Time Policy Enforcement

Beyond batch validation, policies are enforced *continuously* as the engineer edits the topology. Every draft mutation triggers the draft validation function, which evaluates both structural validation rules and organisational policies in a single pass.

Severity-aware findings. Each compliance finding carries a severity level (error, warning, or info). The editor canvas renders findings as spatially-anchored overlays (Figure 21): red glow for errors, amber for warnings. A dedicated compliance panel (Figure 22) lists all active violations with click-to-focus navigation that centres the canvas on the affected node or link.

Finding normalisation. Before UI consumption, findings are deduplicated and sorted by canonical node/link references. When multiple findings affect the same target, the overlay displays only the highest-severity badge, preventing visual clutter while preserving the full finding list in the panel.

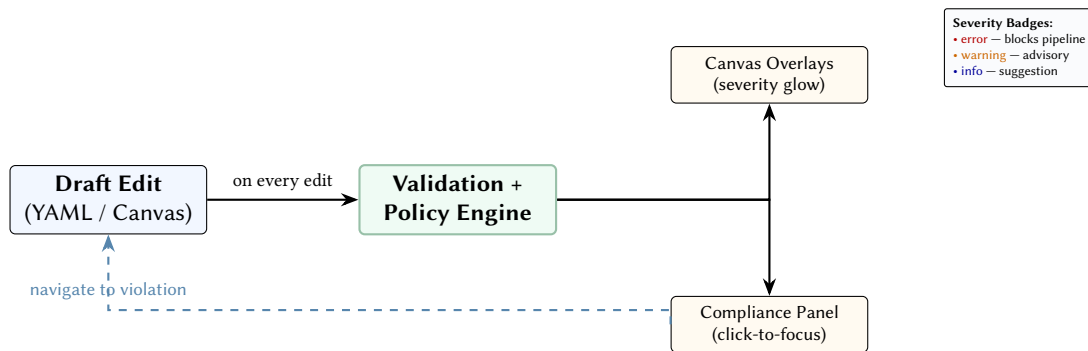


Figure 21. Real-time policy enforcement loop. Every draft edit triggers combined validation and policy evaluation. Findings render as severity-graded canvas overlays and populate a navigable compliance panel.

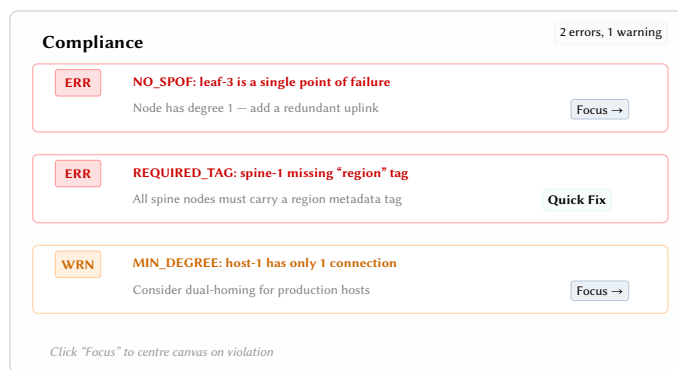


Figure 22. Compliance panel wireframe. Policy violations are listed by severity (error, warning, info) with affected-element references. “Focus” centres the canvas on the offending node; “Quick Fix” applies an automated remediation for deterministic violations.

9.5 Automated Remediation

For deterministic policy violations, the platform offers one-click automated repair (Figure 23). When a policy finding includes machine-readable remediation metadata, the compliance panel displays a “Quick Fix” button alongside the violation description.

Remediation pipeline:

1. The frontend calls the remediation endpoint with the finding identifier.
2. The backend applies the remediation to an in-memory copy of the topology draft (e.g. adding a missing required-tag to a node’s metadata).
3. A fresh validation pass runs against the modified draft, confirming the fix resolves the violation.
4. The patched draft and updated validation results are returned in a single response, enabling the UI to update atomically.

Scope: Currently, only required-tag violations are auto-fixable. Other finding types include explanation-only remediation guidance (human-readable text describing what to change). The remediation metadata format is extensible — new auto-fix generators can be registered per rule type.

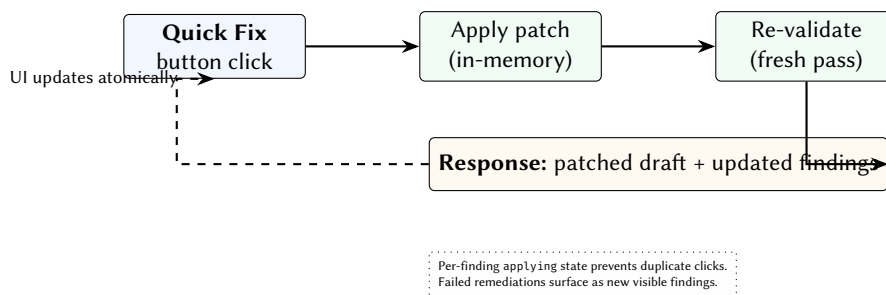


Figure 23. Automated remediation flow. The backend applies the fix in-memory, re-validates, and returns both the patched draft and updated findings in a single round-trip.

Note

Automated remediation for policy violations is distinct from the dissonance remediation described in §5. Policy remediation operates on the *design-time* topology draft; dissonance remediation operates on *live* container configurations. Both use a registry-based dispatch pattern but target different lifecycle stages.

10 Chaos Engineering

The chaos subsystem enables fault injection directly from the topology canvas, continuous resilience scoring, and real-time visualisation of rerouting behaviour during impairment.

10.1 Canvas-Integrated Fault Injection

Operators right-click a node or link on the canvas to trigger chaos actions (reboot, link cut, link restore), as shown in Figure 24. The chaos hook applies optimistic UI updates: the impaired-nodes and impaired-links sets are updated *before* the API call completes, providing instant visual feedback. On error, the update is rolled back.

Actions are dispatched to the ContainerLab backend (§8), which executes the corresponding Docker operations (container restart for reboot, interface shutdown for link cut). A 10-second auto-clear timer handles transient reboot states.

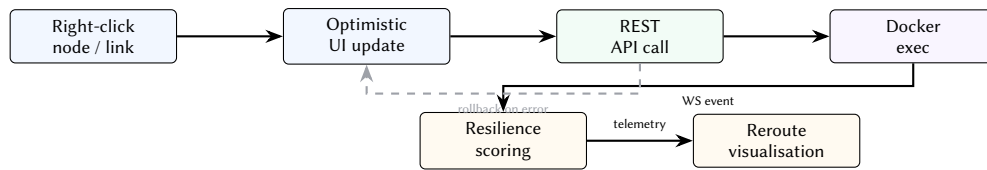


Figure 24. Chaos engineering lifecycle. Optimistic UI updates provide instant feedback; the backend dispatches Docker operations; WebSocket events trigger resilience scoring and reroute visualisation. On API error, the optimistic update is rolled back.

10.2 Resilience Scoring

The resilience scorer continuously measures network health by comparing baseline telemetry (§16) against the impaired state:

$$\text{overall} = 0.5 \cdot \text{connectivity} + 0.3 \cdot \text{stability} + 0.2 \cdot \text{convergence}$$

where:

Connectivity: $\min\left(\frac{\text{current_bps}}{\text{baseline_bps}} \times 100, 100\right)$ — ratio of current aggregate throughput to baseline.

Stability: $100 - \frac{\text{dead_links}}{\text{total_links}} \times 100$ — a link is “dead” when its throughput drops >50% from baseline.

Convergence: Currently 100% (placeholder for protocol-event-based tracking).

Scores are persisted to an SQLite table with per-component breakdowns and impaired-element snapshots. The chaos manager orchestrates an async polling loop that samples flow telemetry and writes scores at regular intervals.

The frontend resilience score panel (Figure 25) polls every 10 seconds, displaying a colour-graded overall score (green $\geq 90\%$, amber $\geq 70\%$, red $< 70\%$), per-component breakdowns, active impairment badges, and a downloadable impact report.

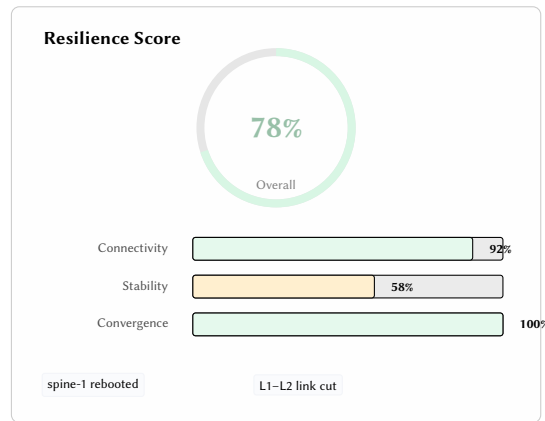


Figure 25. Resilience scoring panel wireframe. The circular gauge shows the weighted overall score; horizontal bars break down connectivity, stability, and convergence components. Active impairment badges identify the chaos actions currently degrading the network.

10.3 Real-Time Reroute Visualisation

When a chaos event is detected (via workflow WebSocket events), the reroute visualiser starts a bounded session:

1. **Trigger detection:** Extract the affected node or link identifier from the event metadata.
2. **Traffic monitoring:** Subscribe to flow telemetry (§16); track the top 6 links by traffic-shift magnitude.
3. **Session termination:** End when traffic shifts stabilise (no new shifts for 2 s after a 3 s search window), a protocol convergence event arrives, or a 20 s hard cap expires.

During the session, an SVG overlay renders animated dashed orange lines along rerouting paths, pulsing search rings on affected nodes, and an elapsed-time badge. The overlay syncs with the Sigma.js canvas transform via the browser’s animation-frame API for smooth 60 fps updates.

11 Frontend Architecture

The ANK Workbench frontend is built as a highly interactive, panel-driven React workspace. The main viewport hosts the topology canvas, a React wrapper around Sigma.js for GPU-accelerated graph rendering. It is designed to accommodate complex operations (code editing, visual graph manipulation, terminal inspection, and data analysis) simultaneously without cognitive overload.

11.1 Projects Landing Page

The entry point to the application is a card-grid landing page (Figure 26). Each project is rendered as a showcase card displaying the project name, version badge, description (3-line clamp), filtered “key insight” tags, resilience score (colour-coded: green $\geq 90\%$, amber $\geq 70\%$, red $< 70\%$), convergence time, author, and relative timestamp. Cards lift on hover with an accent-glow shadow to provide interactive affordance.

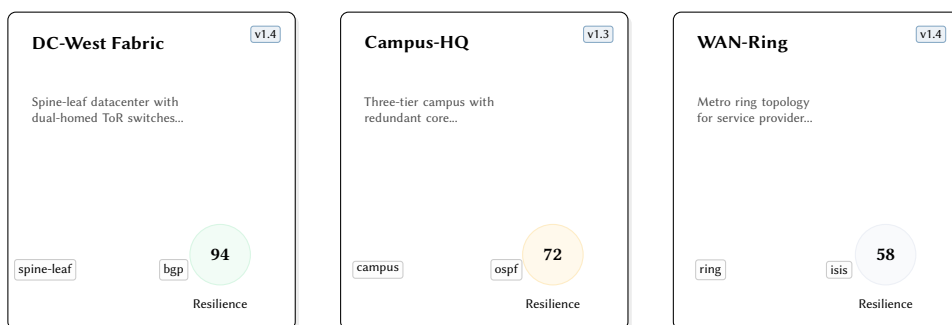


Figure 26. Projects landing page with ShowcaseCard components. Each card displays metadata, version, resilience score (colour-coded), and topology tags.

11.2 Workspace Layout

The workspace (Figure 27) uses an orthogonal resizable-panel layout with three zones: a left workflow panel, the main viewport, and a right context panel.

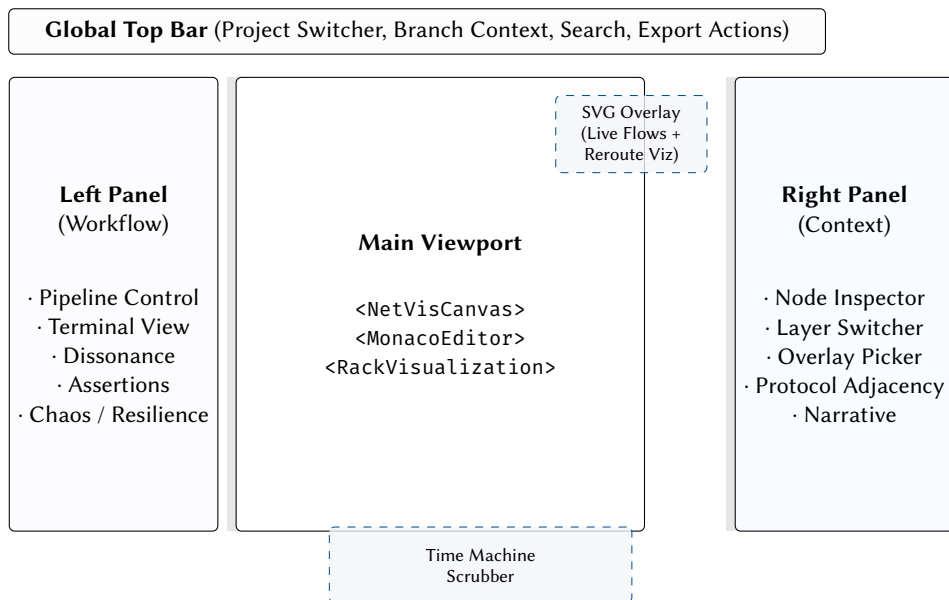


Figure 27. The workspace layout features orthogonal resizing panels. The main viewport hosts the Sigma.js-based NetVisCanvas (or alternative views: Monaco editor, rack layout), stacked with transparent SVG overlays for live data visualisation.

11.3 Overlay Compositor

The topology canvas supports composable overlay layers (Figure 28), computed in a single memoised pass. A layer-mode switcher (Physical / Logical / Protocol) controls which subset of layers is active:

1. Base style (node type colours)
2. Layer visibility (hide nodes/links not in the active layer)
3. Impairment metadata (from deployment state)
4. Simulation overlays (link utilisation, protocol state, routing tables)
5. Flow path tracing (hop-by-hop highlighting from flow statistics)
6. Dissonance status
7. Graph metrics (centrality, community, resilience)
8. PCAP / live telemetry (per-protocol colour coding)
9. Time Machine highlights
10. Reroute visualisation (animated dashed paths during chaos)
11. Terminal hover indicator
12. Chaos state (highest priority — always wins)

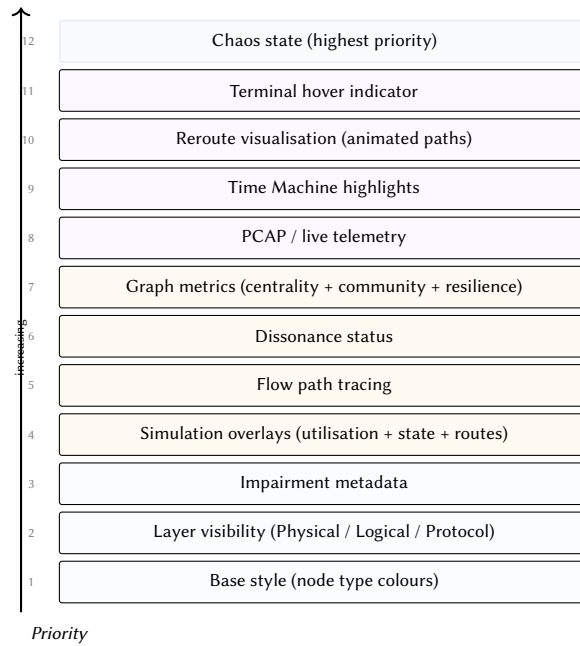


Figure 28. Overlay compositor stack. Each layer is computed in a single useMemo pass. Higher-priority layers override lower ones. The LayerMode switcher controls which subset is active.

Stale-dimming: When the topology hash or artifact hash diverges from the current state, all overlays are automatically dimmed to $\alpha = 0.55$. This communicates “these results may be outdated” without adding a blocking modal.

Rationale: We tried modal warnings first. Users dismissed them immediately and then acted on stale data. Subtle visual degradation (reduced opacity) proved more effective — the canvas looks “wrong” enough to prompt investigation without interrupting the workflow.

11.4 Time Machine

The simulation Time Machine allows operators to scrub through protocol convergence events at microsecond granularity.

Persistence layer: Temporal traces are stored in IndexedDB with two object stores: an events store (indexed by timestamp for range queries) and a keyframes store (periodic full-state snapshots). This enables offline replay and survives page refreshes.

State reconstruction: Given a target timestamp t , the reconstruction hook finds the nearest keyframe k where $k.ts \leq t$, then applies all subsequent patch events up to t . Full-state events replace the accumulator entirely. This keyframe-plus-replay strategy bounds reconstruction cost to the number of events between keyframes rather than the full trace length.

Two-phase fetch strategy:

1. **Bounds query:** Fetch a single event to discover the temporal range $[t_{\min}, t_{\max}]$.
2. **Windowed query:** Fetch events within $[\text{cursor} - 250\text{ms}, \text{cursor} + 250\text{ms}]$ as the user moves the scrubber.

This avoids loading the entire trace into the browser while enabling smooth interactive scrubbing. Both queries use request cancellation when the cursor moves before a response arrives.

When the simulator does not emit native microsecond traces, the system synthesises a coarse trace from protocol-level events and displays a visual “coarse” badge. Figure 29 shows the scrubber interface.

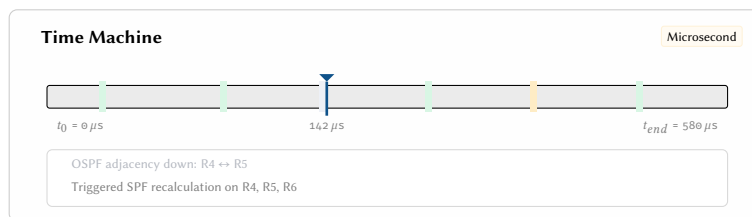


Figure 29. Time Machine scrubber wireframe. Coloured markers on the timeline indicate protocol events (green = convergence, red = failure, amber = state change). The playhead can be dragged to reconstruct network state at any point via keyframe-plus-replay.

11.5 Live Packet Trace Visualisation

To achieve smooth 60 fps animations of real-time packet flows across large topologies, the frontend bypasses the standard React rendering lifecycle and the Sigma.js canvas layer entirely. The packet trace overlay sits above the topology canvas as a transparent, pointer-event-disabled SVG layer.

Architecture:

1. **Telemetry Ingestion:** A WebSocket connection streams high-frequency per-link telemetry updates (bytes/sec, protocol breakdown) directly from the simulation engine.
2. **Direct DOM Mutation:** Instead of triggering React re-renders for every camera movement or physics tick, an animation-frame loop reads the latest node world-coordinates directly from the Sigma.js adapter and mutates pre-allocated SVG line elements via refs.
3. **CSS Animation Pipeline:** The packet “pulses” are purely CSS-driven via dash-array keyframes. The animation duration and line width dynamically scale with the link’s throughput telemetry value.
4. **Vector Effect:** A non-scaling-stroke attribute ensures that as the user zooms into the canvas, stroke widths and packet dashes maintain consistent screen-space thickness rather than ballooning arbitrarily.

This hybrid approach (Figure 30) keeps the heavy graph physics and node rendering in the Sigma.js WebGL layer, while the high-frequency temporal data leverages the browser’s hardware-accelerated CSS compositor, resulting in zero JavaScript-driven garbage collection overhead during animation playback.

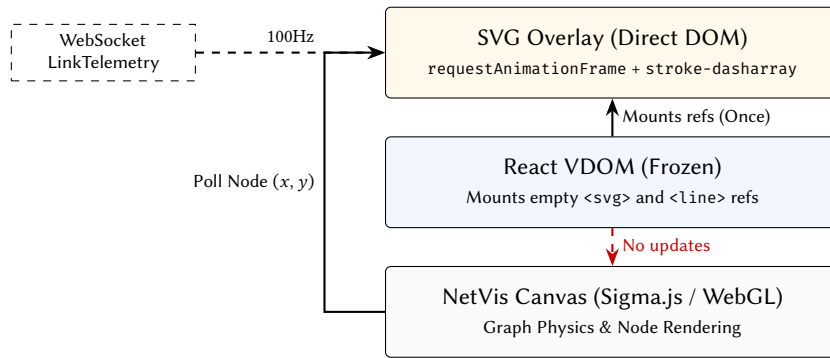


Figure 30. Hybrid SVG-over-Canvas rendering architecture for 60 fps packet trace animations.

11.6 Chaos Engineering Panel

The chaos panel is described in detail in §10. From the frontend perspective, chaos state is rendered as the highest-priority overlay layer, ensuring impaired elements are always visible regardless of which analytical overlay is active.

11.7 Narrative Recording

The narrative system captures annotated sequences of topology visualisation states for documentation and knowledge sharing.

Recording mode: The operator adds chapters, each capturing the current canvas viewport (camera x , y , zoom ratio), Time Machine cursor position (μs), active overlay set, and freeform Markdown commentary.

Playback mode: Chapters are listed with expandable details. Selecting a chapter restores the captured viewport, overlays, and temporal position.

Persistence: Narratives (Figure 31) are stored as an array in project metadata, persisted via debounced saves.

Deep-linking: Each chapter produces a shareable URL with query parameters encoding the narrative and chapter index, enabling direct navigation to a specific visualisation state.

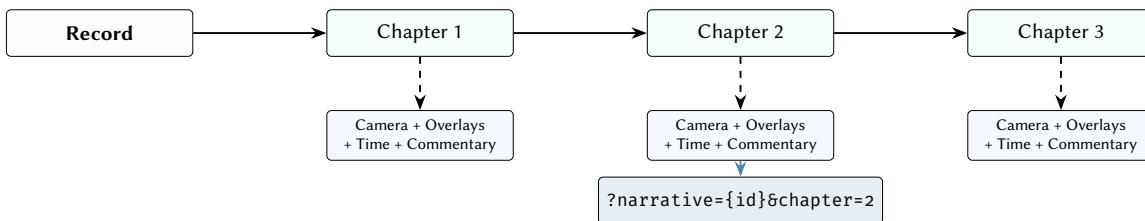


Figure 31. Narrative recording captures per-chapter UI state (camera, overlays, temporal position) and generates deep-linkable URLs for each chapter.

11.8 Protocol Adjacency Panel

When an operator selects a link, the protocol adjacency panel slides in to display routing tables for both endpoint nodes side-by-side. It consumes the per-node routes and protocol state sections of the simulation artifact, normalising flexible field names into a uniform display. Each node heading shows a protocol-state badge (e.g. “ESTABLISHED”, “FULL”).

11.9 Rack Visualisation

The rack visualisation component (Figure 32) provides a data-centre rack management view with drag-and-drop device placement.

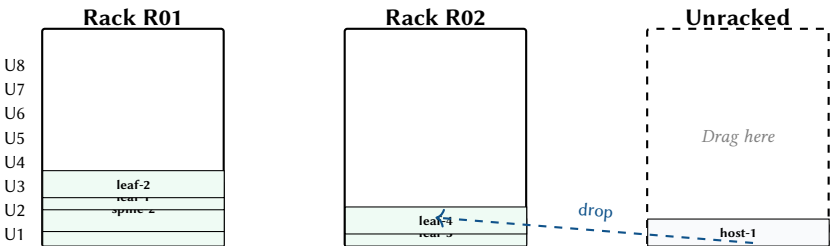


Figure 32. Rack visualisation with drag-and-drop RU assignment. Devices not yet placed appear in the “Unracked” column. Dropping onto a rack assigns rack and ru metadata to the node.

Each rack renders 42 RUs (standard height) in reverse order (U42 at top). Devices are dragged between racks or from the “Unracked” zone using the native browser drag-and-drop API. Moves persist rack and RU metadata to the topology model.

11.10 Topology YAML Panel

The topology YAML panel embeds a Monaco editor configured for YAML syntax, providing inline editing of the raw topology definition. Edits flow through the draft synchronisation hook, which debounces validation (400 ms) and auto-saves to the backend. When a pipeline run is in progress, the editor is locked to prevent mid-run modifications.

11.11 A/B Simulation Comparison

The workflow system (§6) supports comparative analysis across scenario branches. Two completed simulation runs are submitted to the comparison endpoint, which returns per-metric deltas (absolute and percentage change). Figure 33 illustrates the resulting side-by-side view.

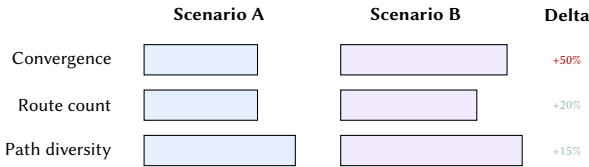


Figure 33. A/B comparison wireframe. Per-metric bars show absolute values for each scenario, with percentage deltas highlighted.

This enables engineers to quantify the impact of topology changes (e.g. adding a redundant link, changing a protocol) on convergence time and routing stability before committing to a design.

11.12 Interactive Terminal

The workspace provides interactive shell access to network devices via an integrated terminal. Two backends are supported: Unix Domain Socket connections to simulator device consoles, and Docker exec sessions into ContainerLab containers with PTY allocation. Figure 34 shows the architecture.

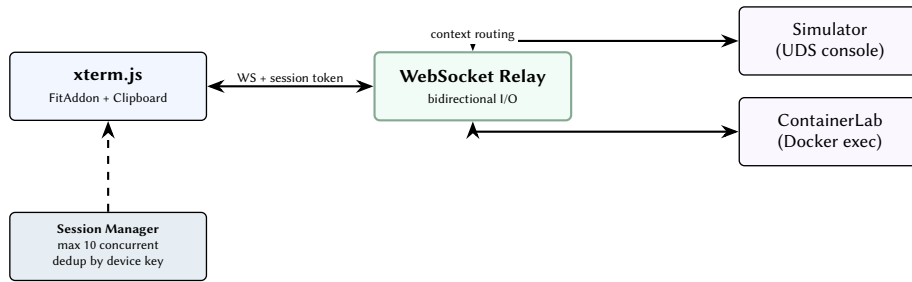


Figure 34. Terminal architecture. The Websocket relay bridges xterm.js to either a simulator UDS console or a ContainerLab Docker exec session, with session deduplication and signed token authentication.

Session management. A singleton session manager deduplicates connections: opening a terminal to an already-connected device focuses the existing session rather than creating a duplicate. A global cap of 10 concurrent sessions prevents resource exhaustion. Sessions disconnect automatically after 10 minutes of inactivity, with a 60-second warning.

Terminal linking. The terminal output is scanned for IP addresses and device names in real time. Detected tokens are rendered as clickable links: hovering highlights the corresponding node on the topology canvas, and clicking navigates to it. IP detection handles both IPv4 and IPv6 (including bracketed notation), and an IP-to-node index built from the topology YAML resolves addresses to device identifiers.

11.13 Scale Visualisation

For topologies exceeding 2,000 nodes or 10,000 edges, the platform switches to a scale-optimised rendering mode that maintains interactive frame rates through three techniques (Figure 35).

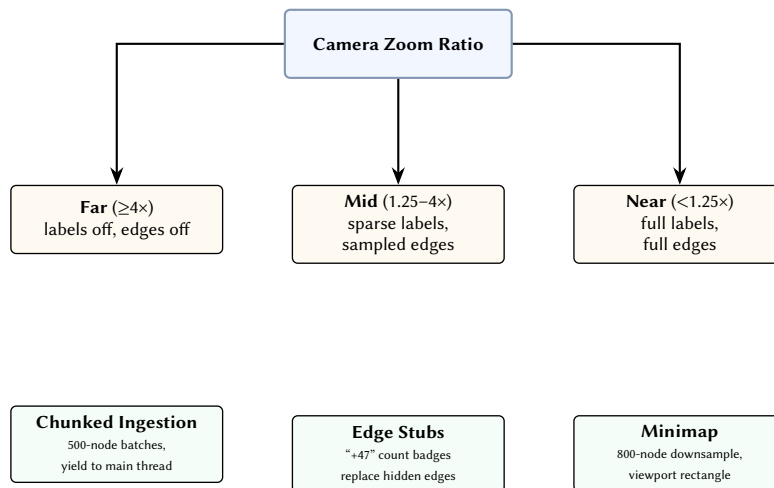


Figure 35. Scale rendering strategy. Camera zoom drives level-of-detail decisions for labels and edges. Chunked graph ingestion prevents main-thread blocking; edge stubs summarise hidden connections; a downsampled minimap provides navigation context.

Level of detail. The camera zoom ratio drives a four-tier LOD system. At the furthest zoom, all labels and edges are hidden; at intermediate zoom, edges are deterministically sampled (every third edge, via stable FNV-1a hash) and labels are sparsified by grid-based culling. Only at close zoom is the full graph rendered.

Chunked ingestion. Graph data is ingested in batches of 500 nodes and 2,000 edges, yielding to the main thread between batches to keep the UI responsive. An abort signal allows cancellation when the user navigates away.

Edge stubs. When edges are hidden at scale, stub nodes are placed near high-degree nodes showing the count of hidden connections (e.g. “+47”), giving the user a sense of connectivity density without rendering thousands of edges.

Search index. A multi-field search index supports queries across node IDs, labels, IP addresses, tags, and roles with ranked scoring. Exact matches on IDs score highest; substring matches on roles score lowest. Multi-token queries require all tokens to match the same node.

12 Topology Import Pipeline

The import subsystem discovers network topology from external sources and converts it into ANK's topology model via a preview-then-commit workflow (Figure 36). Four provider plugins share a common interface:

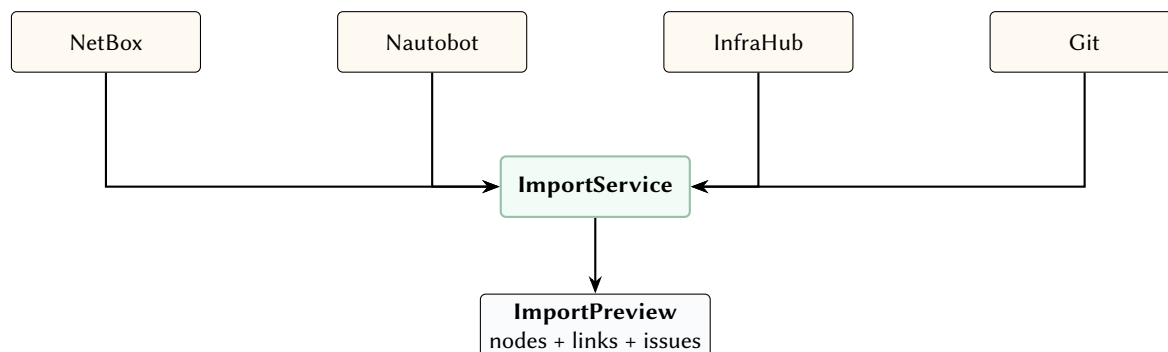


Figure 36. Multi-source topology import architecture. Each provider plugin implements `fetch_preview(url, token, scope)` and returns a common `ImportPreview` structure.

NetBox / Nautobot: Queries the DCIM API for devices (optionally filtered by site), enumerates interfaces and IP addresses, and discovers cables to build the link set. IP addresses are converted from CIDR to dotted-decimal subnet masks.

InfraHub: Uses the async GraphQL SDK to fetch device objects, lazy-loads interface and IP-address relationships, and discovers cable endpoints.

Git: Clones a repository (shallow, with token-based auth), locates the topology YAML file (at the specified path or via recursive glob), and validates it.

All providers return an import preview containing discovered nodes, links, and any issues (e.g. missing interface IPs, dangling cable endpoints). The operator reviews the preview before committing to project creation.

13 GitOps Export

The GitOps subsystem enables teams to push validated network designs to a Git repository and optionally generate CI/CD pipeline files.

13.1 Export Pipeline

The export endpoint accepts the target repository URL, branch, path, access token, commit message, and an optional CI/CD platform (GitHub Actions, GitLab CI, Ansible, or Terraform). Figure 37 shows the four-step pipeline.

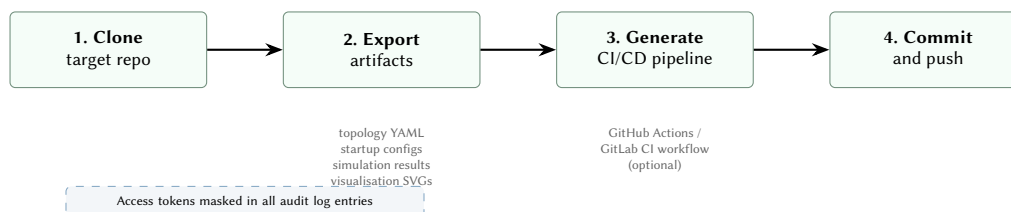


Figure 37. GitOps export pipeline. The service clones, exports validated artifacts, optionally generates a CI/CD workflow, and pushes to the remote.

13.2 HLD Document Generation

The High-Level Design exporter produces a Markdown document with embedded base64 SVG diagrams, a reachability matrix (top 10 nodes by route count), A/B scenario comparison tables (when multiple scenarios exist), predictive risk findings, and implementation notes.

14 CI/CD Feedback Integration

The CI/CD feedback subsystem closes the loop between GitOps export and production deployment pipelines. When a user pushes a validated topology with CI/CD generation enabled, the platform subscribes to webhook events from the target CI/CD system and surfaces pipeline status back into the workspace.

14.1 Feedback Loop

Figure 38 shows the bidirectional flow: the Workbench pushes topology artifacts and a generated pipeline definition to a Git repository, then receives webhook callbacks as the pipeline executes.

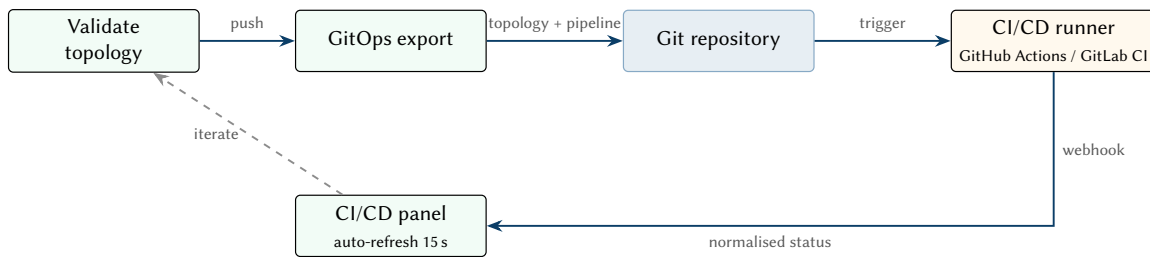


Figure 38. CI/CD feedback loop. Validated topologies are pushed with a generated pipeline; webhook callbacks deliver normalised status back to the workspace panel.

14.2 Webhook Security and Normalisation

Inbound webhooks are authenticated before parsing: GitHub payloads are verified via HMAC SHA-256, GitLab via a shared token header. Figure 39 shows how provider-specific statuses are mapped to a five-state model and matched against active subscriptions.

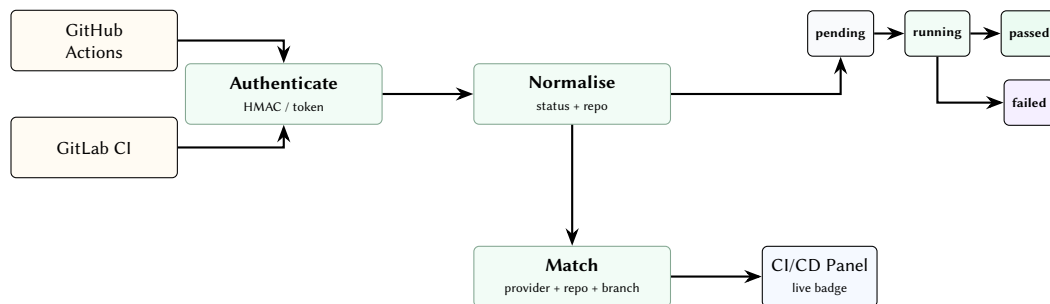


Figure 39. Webhook normalisation. Provider-specific payloads are authenticated, mapped to a five-state model, and matched against active subscriptions by provider, normalised repository, and branch.

14.3 Subscription Matching

When a GitOps push includes a CI/CD platform, the export handler persists a feedback subscription (provider, repository, branch). Each inbound webhook is matched against all active subscriptions by exact provider, normalised repository, and branch. Matched projects receive the normalised result (status, commit SHA, pipeline URL, workflow name).

The CI/CD panel (Figure 40) in the workspace shows the live status badge, subscription details, and a direct link to the external pipeline run. The export manager generates platform-specific pipeline definitions (GitHub Actions, GitLab CI, Ansible, Terraform) that validate the exported topology.

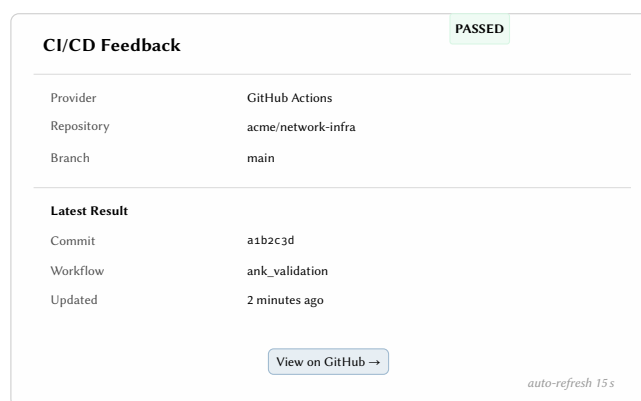


Figure 40. CI/CD feedback panel wireframe. The panel shows normalised pipeline status, subscription metadata, and a link to the external run. Status badges are colour-coded: green (passed), amber (running), red (failed).

15 Real-Time Collaboration

Multiple engineers can edit the same project concurrently via CRDT-based state synchronisation powered by Yjs.

15.1 Architecture

Figure 41 shows the collaboration data flow. The backend collaboration hub is deliberately thin — it relays binary Yjs frames between peers without inspecting the payload, keeping all merge logic in the client-side CRDT.

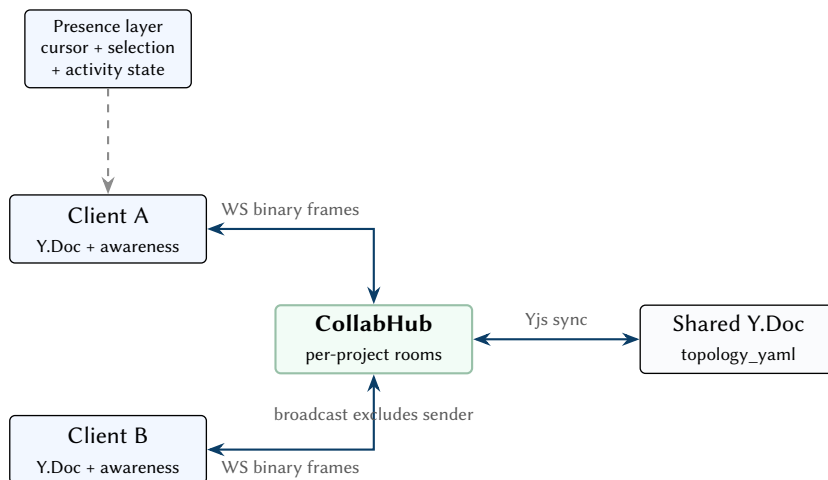


Figure 41. Collaboration architecture. The CollabHub relays binary Yjs frames between project-scoped rooms. Each client maintains a local Y.Doc replica; merges happen via CRDT.

15.2 Draft Synchronisation

The draft synchronisation hook maintains a two-way binding between the local draft YAML and the shared CRDT text. On mount, a bootstrap decision seeds the shared document if it is empty, adopts the remote version if it differs (remote wins), or no-ops if already in sync. Ongoing edits flow in both directions, with an origin tag preventing circular updates — each side ignores changes that it originated. Figure 43 shows the multi-user editing concept.

15.3 Collaborative Undo

A per-user undo manager tracks only locally-originated changes, so undo never reverts a peer's edits. A weak-reference cache ensures a single manager instance per shared text document.

15.4 Presence and Awareness

Figure 42 illustrates the presence data carried by the Yjs awareness protocol.

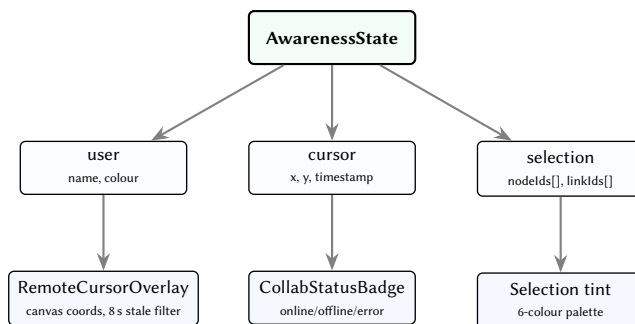


Figure 42. Presence data model. Each peer broadcasts user identity, cursor coordinates, and topology selection. The UI renders remote cursors, a connection badge, and colour-coded selection highlights. Cursor updates are throttled to 80 ms.

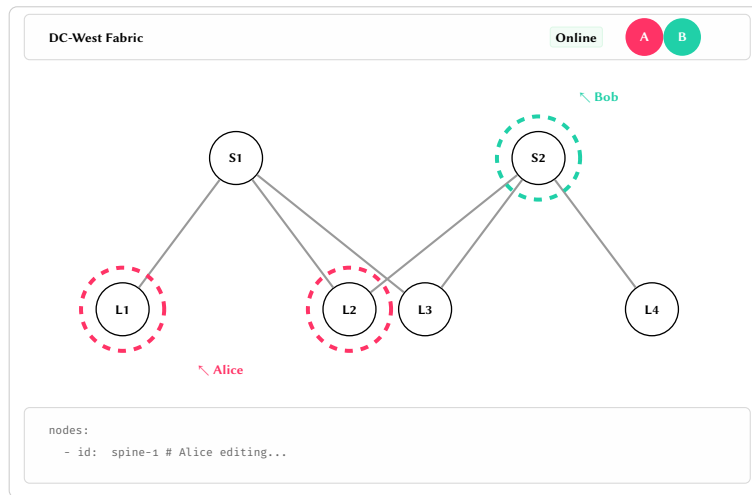


Figure 43. Multi-user collaboration concept. Alice (pink) has selected two leaf nodes; Bob (teal) is inspecting a spine. Presence avatars and a connection badge appear in the toolbar. The shared YAML panel at the bottom updates in real time via CRDT synchronisation.

16 Live Telemetry Ingestion

ANK Workbench can ingest live protocol and topology state from operational networks, enabling engineers to compare the design-time model against production reality.

16.1 Ingestion Pipeline

Figure 44 shows the end-to-end flow from an operational network collector through to the workspace UI.

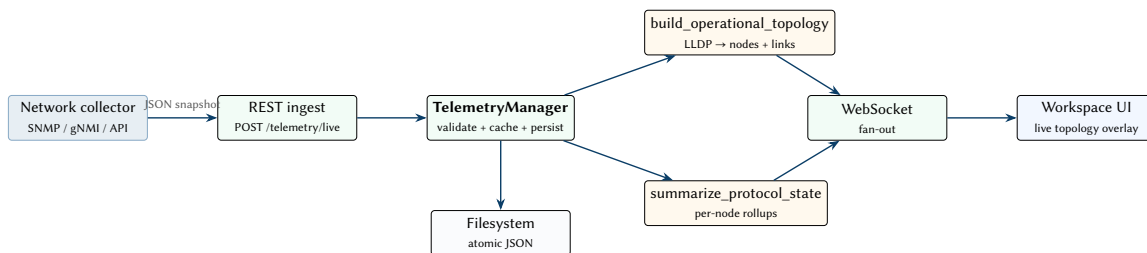


Figure 44. Live telemetry pipeline. Snapshots from network collectors are ingested via REST, validated and persisted by the manager, materialised into an operational topology and protocol rollups, then streamed to connected clients over WebSocket.

16.2 Snapshot Data Model

A live telemetry snapshot captures a point-in-time view of the network. Figure 45 shows the object hierarchy.

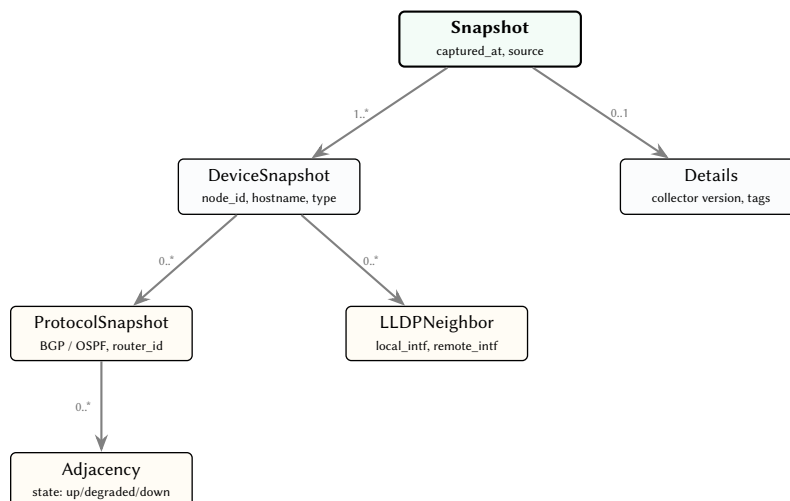


Figure 45. Live telemetry data model. Each snapshot contains device records with protocol adjacency states and LLDP neighbours. Strict validation enforces identifier format, timestamp plausibility, and uniqueness constraints.

Two pure functions derive higher-level artefacts from a raw snapshot:

Operational topology construction Walks LLDP neighbour tables to construct an operational topology graph. Bidirectional links are deduplicated using sorted node-pair keys; self-loops and references to missing neighbours are discarded.

Protocol state summarisation Rolls up adjacency counts per node per protocol into a summary (total, up, degraded, down).

16.4 Streaming

The dedicated telemetry WebSocket endpoint delivers snapshots in real time. On connection (authenticated via a signed session token, 600 s expiry) the server sends the current snapshot immediately if one exists, or a waiting status. Subsequent snapshots are pushed as they arrive via per-subscriber async queues. The frontend renders the data as a colour-coded overlay (Figure 46).

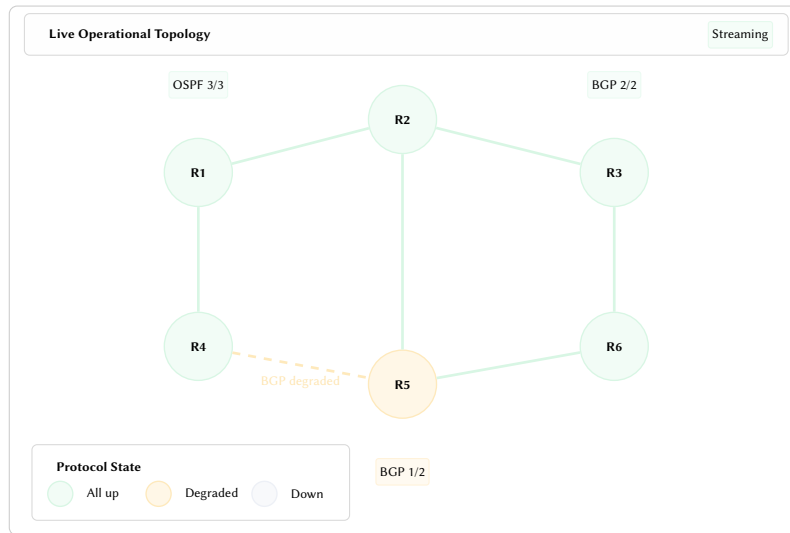


Figure 46. Live telemetry overlay concept. Nodes are colour-coded by protocol health (green = all adjacencies up, amber = degraded, red = down). Per-node badges show adjacency rollups (e.g. “BGP 1/2” means one of two BGP peers is up). Links reflect the worst adjacency state between endpoints.

16.5 Hardware Telemetry Collector

In addition to accepting telemetry snapshots from external collectors, the platform includes a built-in hardware polling subsystem that connects directly to network devices via SSH. Figure 47 shows the collection pipeline.

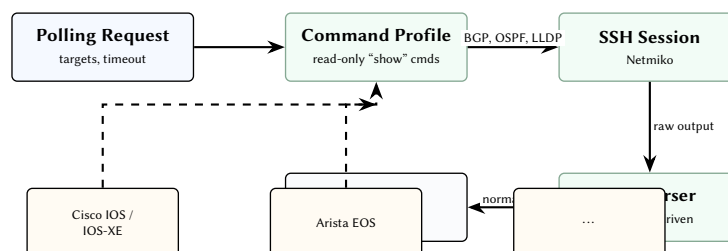


Figure 47. Hardware telemetry collection pipeline. Per-platform command profiles define read-only show commands; TTP templates parse vendor-specific output into normalised protocol snapshots.

Command profiles. Each supported platform (Cisco IOS, Cisco IOS-XE, Arista EOS) defines an immutable command profile specifying which show commands to execute for each protocol family (BGP, OSPF, LLDP). Profiles enforce read-only safety: any command containing mutating tokens (configure, reload, shutdown, etc.) is rejected at validation time.

Parsing. Raw command output is normalised via TTP (Template Text Parser) templates. Vendor-specific state strings are mapped to a canonical three-state model (up, degraded, down) – for example, BGP “Established” maps to up, OSPF “FULL” maps to up, and “2WAY” maps to degraded.

Error handling. Collection runs per-device with independent error isolation. When a device is unreachable or a command fails, the error is captured without aborting the remaining targets. An optional partial-failure mode allows the caller to accept incomplete snapshots when some devices succeed.

17 MCP Server Integration

The platform exposes its capabilities to AI assistants via a FastMCP server that communicates over stdio transport. Figure 48 shows how AI assistants interact with the platform through MCP tools and resources.

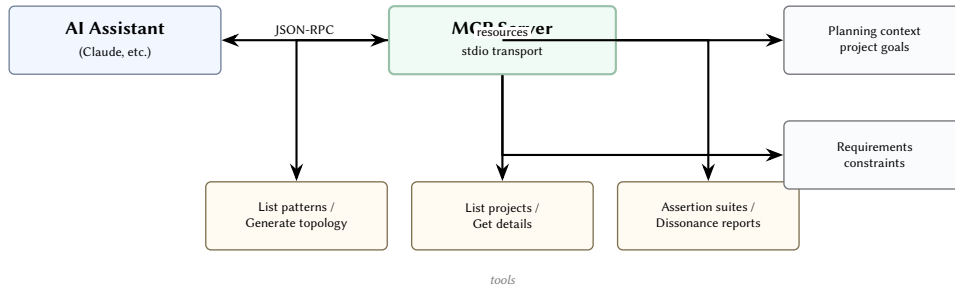


Figure 48. MCP server integration. AI assistants access platform capabilities (topology generation, project inspection, assertion suites, dissonance reports) and planning context via the MCP protocol over stdio.

18 Observability and Flow Analysis

The observability subsystem captures, enriches, and visualises network flow data and protocol events across both simulated and live environments. It bridges the gap between coarse simulation artifacts and fine-grained operational telemetry.

18.1 Architecture

Figure 49 shows the two data paths: real-time NetFlow/IPFIX collection from running networks, and protocol event extraction from simulation artifacts.

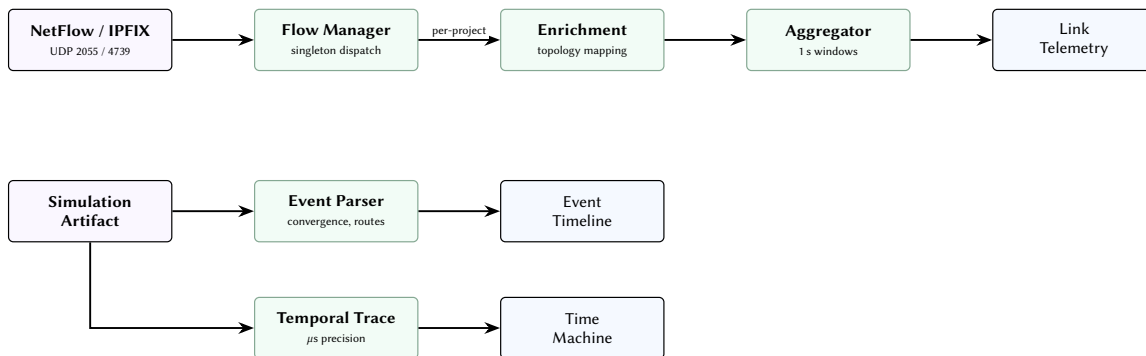


Figure 49. Observability architecture. Real-time flows are enriched with topology metadata and aggregated into per-link telemetry. Simulation artefacts yield protocol events and temporal traces.

18.2 Flow Collection and Enrichment

A pair of async UDP listeners (NetFlow v9 on port 2055, IPFIX on port 4739) start with the application and feed a singleton flow manager. The manager routes incoming records to project-scoped handlers by mapping source IPs (management interfaces) to project and node identifiers.

Each project handler enriches raw flows with topology metadata: SNMP interface indices are resolved to interface names, source–target pairs are mapped to topology links, and protocol is classified by heuristic (port numbers, IP protocol field). Enriched flows are then aggregated into one-second windows, producing per-link telemetry summaries (bytes/sec, packets/sec, protocol breakdown) that feed the canvas overlays and the live packet trace visualisation.

18.3 Protocol Event Extraction

When a simulation run completes, the event parser extracts structured protocol events from the artifact’s convergence metadata, command results, and routing tables. Events carry a protocol tag (OSPF, BGP, ISIS, or system), severity level, originating device, and human-readable summary.

The frontend event timeline offers three views — chronological, grouped by protocol, and grouped by device (swimlane) — with filtering by protocol and free-text search. Clicking an event highlights the corresponding node on the topology canvas.

18.4 Temporal Tracing

When the simulator emits native microsecond-precision traces, they are stored as temporal trace artifacts. When native traces are unavailable, the system synthesises a coarse trace from protocol-level events and marks it with a visual “coarse” badge. Both paths feed the Time Machine scrubber (Section 11).

19 Global Search

A pluggable search service (Figure 50) enables cross-cutting queries across projects, topology nodes, and configuration files.

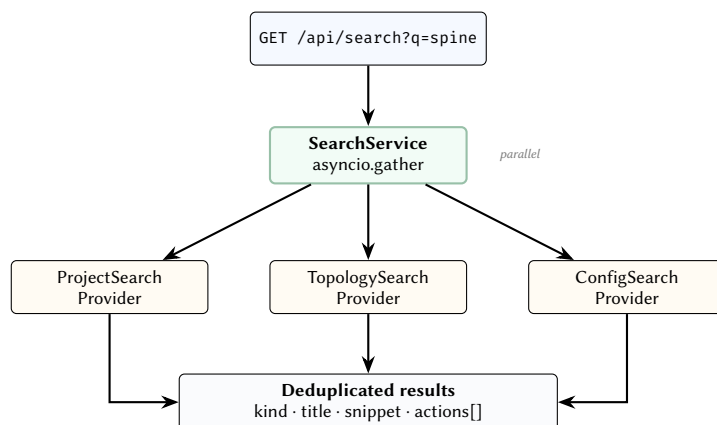


Figure 50. Global search architecture. Three provider plugins run in parallel; results are deduplicated and returned with actionable handlers (navigate, run_sim, open_terminal, inspect).

Each result includes a kind, title, snippet, link, and an actions array with handler types. Results are deduplicated across providers; minimum query length is 2 characters. The provider interface is designed for extensibility — adding a new searchable domain requires implementing a single async method.

A full API reference (REST endpoints and WebSocket events) is provided in Appendix A.

20 Audit Trail

Every state-changing operation in the platform is recorded as an immutable audit event. This trail serves two purposes: compliance visibility for multi-user teams, and forensic replay when investigating unexpected configuration drift.

20.1 Event Model

Each audit entry captures the *who*, *what*, *where*, and *when* of an action: the authenticated user (name, role, and client IP), the action type, the affected resource (type and identifier), and a millisecond-resolution timestamp. An optional payload carries action-specific metadata—for example, a pipeline completion event includes the workflow run identifier so that a reviewer can teleport directly to the corresponding run log.

Action types span the full lifecycle of platform objects: project creation and deletion, topology updates and exports, pipeline step transitions, simulation start/stop/reset, ContainerLab deploy and destroy, scenario branching (create, rename, activate, delete, restore, merge), policy edits, and cross-scenario comparisons. In total, approximately thirty discrete action types are defined.

20.2 Persistence and Indexing

Events are persisted to an SQLite table with three covering indexes—by timestamp, by originating user, and by resource type–identifier pair. These indexes support the three most common query patterns: chronological browsing, per-user activity review, and resource-level change history. The payload is stored as serialised JSON, keeping the schema flexible as new action types are introduced.

20.3 Access Control

The audit trail is exposed through two scoped endpoints. A *system-wide* endpoint returns events across all projects and is restricted to administrators; it supports filtering by user, resource type, and resource identifier, with configurable pagination (up to 500 entries per page). A *project-scoped* endpoint returns events for a single project and is available to designers, approvers, and administrators—viewers are excluded to prevent information leakage about actions they cannot perform.

20.4 Frontend Presentation

The audit trail is presented in two contexts (Figure 51). The *admin audit page* displays the system-wide stream with column sorting and filter controls. The *project history panel* shows a project-scoped timeline embedded in the project workspace.

Both views render each event as a table row with an icon indicating the action category (e.g. a flask for simulation events, a rocket for ContainerLab deploys), the acting user with a role-coloured badge, the affected resource with a truncated identifier tooltip, the formatted timestamp, and the originating IP address. When an

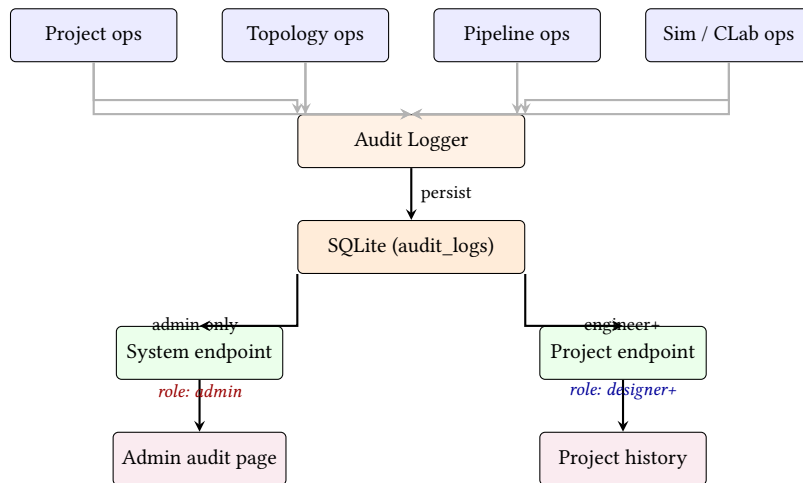


Figure 51. Audit event flow. Every state-changing operation is captured by the audit logger, persisted to SQLite, and surfaced through role-gated endpoints to the admin audit page and per-project history panel.

event’s payload carries a workflow run identifier, a “Teleport” button appears, allowing the reviewer to jump directly to the associated pipeline run without manual navigation.

21 Evaluation

This section reports quantitative characteristics of the platform, measured from the implementation and documented configuration constants. All values are drawn from the codebase and reflect the system as deployed.

21.1 Startup and Convergence Latency

Simulation startup time scales linearly with topology size. The timeout budget allocates a 10 s base (daemon startup), 0.5 s per device (initialisation, interface setup), and 0.1 s per link (ARP resolution, protocol adjacencies), capped at 120 s:

$$T_{\text{sim}} = \min(10 + 0.5N + 0.1L, 120) \text{ seconds}$$

For representative topologies: a 3-device spine-leaf starts in ~ 12 s; a 20-device campus in ~ 24 s; a 100-device fabric in ~ 80 s. Container boot adds a platform-dependent overhead: Arista cEOS images reach readiness in ~ 60 s, Cisco IOS-XR in ~ 180 s, and Nokia SR Linux in ~ 300 s. The readiness monitor polls at 5 s intervals with a 10-minute hard cap.

21.2 Real-Time Communication

The WebSocket event bus allocates a 256-event queue per project subscriber. On overflow, the oldest event is dropped deterministically and the drop is recorded in metadata — this prevents a slow consumer from blocking other subscribers. Session tokens expire after 600 s; terminal WebSocket connections use exponential backoff (base 1 s, cap 30 s, $\pm 10\%$ jitter) with up to 10 reconnection attempts.

Flow telemetry is aggregated in 1 s windows before broadcast. Collaborative presence updates are throttled to 80 ms intervals, with remote peers timed out after 8 s of inactivity. System status polling runs at 30 s intervals.

21.3 Scale Boundaries

Table 4. Operational scale boundaries.

Dimension	Threshold	Behaviour
Topology rendering	2 000 nodes	Switch to LOD mode
Edge rendering	8 000 edges	Hide edges, show stubs
Graph ingestion	500 nodes/batch	Chunked with yield
Edge ingestion	2 000 edges/batch	Chunked with yield
Search results	200 items	UI cap (prevents freeze)
Geometry cache	10 topologies	LRU eviction
Force layout iterations	300	Convergence target

The benchmark suite includes three 10 000-node topologies (sparse random, dense random, and hub-heavy preferential attachment), all deterministically seeded for reproducible testing.

21.4 API Pagination and Rate Limits

Table 5. API pagination defaults and ceilings.

Endpoint	Default	Maximum
System audit logs	100	500
Project audit logs	50	200
Pipeline logs	50	200
Workflow events	500	2 000
Project listing	50	200
Resilience history	50	—

All SQLite connections use a 30 s timeout. Full-text search uses BM25 ranking for result ordering. The retry policy for transient errors applies exponential backoff (0.1–2.0 s) with a maximum of 3 attempts.

21.5 Validation Constants

Physical cable validation enforces IEEE 802.3 limits: DAC cables ≤ 7 m, copper cables ≤ 100 m, per-rack power ≤ 5 kW. Assertion traceroutes use a 30-hop ceiling (analogous to IP TTL). Dissonance stabilisation polls at 1 s intervals to detect routing table convergence before comparison.

22 Related Work

Network design and validation tools typically address individual stages of the lifecycle. ANK Workbench integrates multiple stages into a single platform; this section positions it against existing tools.

Network emulators. GNS3 and EVE-NG provide GUI-driven emulation using real vendor operating system images. Both excel at interactive lab exercises but lack a declarative topology model, protocol simulation, or dissonance detection. ContainerLab takes a code-first approach with YAML topology definitions and container-based emulation at lower resource cost. ANK Workbench uses ContainerLab as its emulation backend (§8) and adds simulation, dual-backend assertions, policy validation, and chaos engineering on top.

Configuration analysis. Batfish analyses device configurations statically, building vendor-agnostic models to check reachability, loop-freedom, and compliance *before* deployment. It is complementary to ANK Workbench: Batfish validates existing configurations, whereas ANK Workbench validates a *design-time* topology model against both simulated and emulated behaviour. The two could be composed by exporting ANK-generated configurations to Batfish for static analysis.

Network source-of-truth platforms. NetBox and Nautobot maintain inventory, IPAM, and cable-plant records — they are authoritative sources of intended state but provide no simulation, emulation, or testing. ANK Workbench imports from both via the topology import pipeline (§12) and could export operational topology data back, closing a data loop between design validation and production inventory.

Orchestration and IaC. Cisco NSO automates multi-vendor service provisioning with YANG models and transactional rollback. Terraform manages cloud infrastructure declaratively with state tracking. Both focus on *deploying* infrastructure rather than *validating design intent*. ANK Workbench occupies the pre-deployment design-validation space and generates GitOps-compatible artefacts (§13) that such orchestrators can consume.

Positioning. To the best of our knowledge, no existing tool combines declarative topology modelling, dual-backend simulation and emulation with dissonance detection, canvas-integrated chaos engineering, real-time policy enforcement, CRDT-based collaboration, live telemetry overlay, and MCP-driven AI integration within a single platform. The closest overlap is with ContainerLab (emulation) and Batfish (static analysis), each of which addresses a subset of the workflow.

23 Limitations

Simulation fidelity.

Protocol simulators model control-plane behaviour only; data-plane forwarding and vendor-specific quirks are not modelled. Dissonance detection surfaces these gaps but cannot predict them. The dual-fidelity architecture treats this as a feature: ContainerLab serves as the ground-truth environment.

Remediation coverage.

Remediation generation currently covers Arista EOS only. The registry supports arbitrary platforms, but generators must be written per vendor. For unsupported platforms the dissonance report still identifies the required changes. Nokia SR Linux and Cisco IOS-XE generators are planned.

Stabilisation heuristic.

The SHA-based stabilisation loop for FIB collection is heuristic: two consecutive identical hashes do not guarantee protocol convergence. Increasing the poll interval mitigates false stability; nodes marked as not steady are excluded from the diff. Integration with protocol convergence signals (e.g. OSPF adjacency state) is under investigation.

Container readiness.

Vendor NOS images have widely varying boot times (10 s for Linux hosts, 3+ min for cEOS) and no standardised readiness signal. Per-kind strategies are fragile and must be maintained as new image versions are released.

Scale.

The platform has been tested with topologies up to 200 nodes. Behaviour at larger scales (500+ nodes) is untested, particularly for the overlay compositor and the dissonance collection pipeline. Multi-host distribution mitigates single-machine limits.

Resilience convergence scoring.

The convergence component of the resilience score is currently a fixed 100% placeholder. Integrating protocol convergence events from the simulation engine and ContainerLab telemetry is planned.

Import provider maturity.

The four import providers (NetBox, Nautobot, InfraHub, Git) have varying field coverage. Complex NetBox features (virtual chassis, LAG interfaces, VRFs) are not fully mapped. The preview-then-commit workflow lets operators inspect and correct imported topologies before creating a project.

24 Future Work

Expanded remediation coverage: Add generators for Nokia SR Linux, Cisco IOS-XE, and Juniper JunOS.

Incremental dissonance: When a topology change affects only a subset of nodes, re-collect FIB only from affected nodes rather than the full topology.

Convergence-based resilience scoring: Replace the convergence placeholder (currently fixed at 100%) with event-driven tracking using protocol convergence signals from the simulation engine.

Formal evaluation: Conduct user studies with network operators to measure time-to-detect and false-positive rates.

IaC generation: Generate Terraform and Ansible outputs from validated topologies for production deployment.

Expanded MCP tool coverage: Expose chaos actions, policy management, and pipeline execution through the MCP server for full AI-assisted operation.

Design-vs-operational dissonance: Compare the design-time topology model against the live operational topology derived from telemetry ingestion, surfacing drift between intent and reality.

Collaboration conflict resolution UI: Provide a visual merge interface when concurrent edits produce semantic conflicts (e.g. two users adding overlapping IP allocations).

Telemetry-driven alerting: Generate notifications when live protocol adjacency states diverge from the expected topology, enabling proactive incident detection.

25 Conclusion

ANK Workbench demonstrates that the full network design lifecycle — topology modelling, protocol simulation, container-based emulation, assertion-driven validation, and remediation — can be unified in a single web-based platform without sacrificing the fidelity of each stage.

Three architectural choices underpin this integration. First, a *dual-fidelity* execution model treats simulation and emulation as interchangeable backends, with automated dissonance detection bridging the gap between prediction and reality. Second, content-addressed artifacts and a staleness propagation graph keep every pipeline step reproducible and incrementally re-executable. Third, CRDT-based state synchronisation enables real-time multi-user collaboration across all stages of the workflow.

The platform’s policy engine, chaos engineering tools, live hardware telemetry overlay, and MCP-based AI integration extend the design loop beyond static validation into continuous, operational assurance. Together these capabilities address a gap in the current tooling landscape, where emulators, configuration analysers, and source-of-truth systems each cover only a fragment of the lifecycle.

We hope this report provides both a useful reference for adopters and a foundation for the research directions outlined in §24.

A API Reference

ANK Workbench exposes a REST API over HTTP. All endpoints require an API key via the X-API-Key header (configured via ANK_WORKBENCH_API_KEY).

A.1 Core Endpoints

Table 6. Key API endpoints (subset).

Method	Path	Description
GET	/api/projects	List all projects
POST	/api/projects	Create a project
GET	/api/projects/{id}/topology	Get topology YAML
PUT	/api/projects/{id}/topology	Update topology
POST	/api/projects/{id}/validate	Run validation
POST	/api/projects/{id}/simulate	Run simulation
POST	/api/projects/{id}/assertions/run	Run assertion suite
POST	/api/projects/{id}/dissonance/run	Run dissonance comparison
GET	/api/projects/{id}/dissonance/latest	Get latest dissonance report
POST	/api/containerlab/deploy	Deploy ContainerLab topology
DELETE	/api/containerlab/destroy	Destroy deployment
GET	/api/containerlab/status	Get container status
POST	/api/dissonance/{id}/remediate/{node}	Generate remediation commands
POST	/api/dissonance/{id}/apply/{node}	Push remediation to device
POST	/api/projects/{id}/export/git	GitOps push to remote repository
GET	/api/search?q=...	Global cross-project search
GET	/api/projects/{id}/policies	List design policies
PUT	/api/projects/{id}/policies	Upsert a policy
POST	/api/projects/{id}/chaos/{action}	Trigger chaos action (reboot, cut, restore)
GET	/api/projects/{id}/resilience/score	Current resilience score
GET	/api/projects/{id}/resilience/history	Score time-series
GET	/api/projects/{id}/resilience/report	Download impact report
POST	/api/import/preview	Preview topology from external source
POST	/api/workflows/compare	A/B simulation comparison
POST	/webhooks/cicd/github	Receive GitHub Actions webhook
POST	/webhooks/cicd/gitlab	Receive GitLab CI webhook
GET	/api/projects/{id}/export/cicd	Generate CI/CD pipeline definition
POST	/api/projects/{id}/telemetry/live	Ingest live telemetry snapshot
GET	/api/projects/{id}/telemetry/live	Get latest telemetry with protocol summaries
GET	/api/projects/{id}/artifacts/operational-topology/live	LLDP-derived operational topology

A.2 WebSocket Events

The WebSocket endpoint at /api/ws streams structured events:

Table 7. Key WebSocket event types.

Event type	Payload
workflow.step.started	Step name, timestamp
workflow.step.completed	Step name, duration, artifact path
workflow.staleness_changed	Map of step → stale boolean
dissonance.node.completed	Node name, status, diff summary
assertions.assertion.completed	Assertion index, result, output snippet
containerlab.container.ready	Container name, readiness status
containerlab.chaos.*	Chaos trigger with node/link metadata
flow.telemetry	Per-link bytes/sec and protocol breakdown
<i>Dedicated WebSocket endpoints</i>	
/projects/{id}/collab/ws	Yjs binary sync + awareness (collaboration)
/projects/{id}/telemetry/live/ws	Live telemetry snapshot stream

Revision History

Version	Changes
1.2	Live telemetry, collaboration (CRDT), CI/CD feedback, policy enforcement, remediation, flow tracing, audit, observability
1.1	Chaos engineering, resilience scoring, policy engine, GitOps, MCP, topology generation, import pipeline
1.0	Initial release (March 2026)